Evidence, not verdicts: what 590 public PX4 crash logs can and cannot tell a drone fleet operator

Nagesh Chougule[a,*], Vibhav Ambardekar[a], Aditya Pathare[a], Atharv Waghmare[a], Shreyash Pawar[a], Piyush Bhamare[a]

[a]*Department of Mechanical Engineering, COEP Technological University, Pune, India*

Abstract

Flight logs are expected to show whether the pilot or the aircraft caused a drone crash, and which drones will fail next. We tested both on 590 crash logs from a 479,547-flight PX4 archive, declaring confirmatory tests in advance (†marks later exploratory analyses). A fixed rule reached 0.47 balanced accuracy on 112 held-out crashes (pass mark 0.70) and models stayed near chance, also on multicopters; a published log checker did no better. †A blind independent relabelling agreed with the uploaders no better than chance and was not recovered either, so label noise cannot be excluded. †Much apparent crash-detection skill came from the evaluation design: across 32 settings, AUC fell by 0.21 [0.06, 0.34], of which 0.06 came from grouping folds by vehicle, and a re-implemented published fault detector (short of its published accuracy) fell from AUC 0.94 to 0.63 once whole days were held out; before any fault existed it separated fault flights at 0.85 with flights held out, but 0.57 with days held out. †The tilt rule that usually fixes the crash moment misses roughly 7–50% of crashes, depending on the rater, and pilot-rated logs show a tilt event half as often as aircraft-rated ones. Crash flights showed no warning against the drone's own earlier flights. Drones that later crashed vibrated more than matched drones (†$p = 0.012$ by airframe; 0.076 with operator sites weighted equally), but vibration ranking did not clearly beat random inspection. We give a reporting checklist and an evidence-first assurance design.

Keywords: Unmanned aircraft systems, Flight logs, Crash investigation, Fault diagnosis, Data leakage, Fleet assurance

*Corresponding author.
Email address: nkc.mech@coeptech.ac.in (Nagesh Chougule)

1. Introduction

Small unmanned aircraft are now flown commercially for survey, inspection, agriculture and delivery, and the rules that govern them place weight on records. In the European Union and the United Kingdom an operator in the specific category must keep records of operations, maintenance and personnel for at least three years [1, 2]; India requires an accident to be reported within 48 hours [3], and the United States within ten days [4]. When something goes wrong, the onboard flight log is the richest record available. Open-source log viewers, commercial fleet platforms and, more recently, machine-learning and language-model prototypes promise to turn that log into an answer: what failed, and whether the pilot or the aircraft was responsible [5, 6].

The question matters beyond engineering curiosity. In aviation safety science, accidents are recognized as systemic, multi-causal events involving human factors, environmental conditions and organizational influences [7]. Commercial warranty claims, insurance settlements and regulatory filings nevertheless frequently compress these dynamics into a binary decision: whether the pilot or the aircraft was primarily responsible. An automatic verdict that looks objective can carry weight it has not earned. Fleet operators face a related but different question: not why one flight ended badly, but which of their drones are most likely to fail next and should be inspected first. A log-based method that is good at the second task need not be good at the first, and confusing the two can mislead both investigators and maintainers.

Whether logs can deliver on either expectation has, to our knowledge, not been tested at scale against real crashes with evaluation protocols that guard against the two known sources of optimism. First, a log that contains a crash also contains its consequences (saturated motors, extreme attitudes, estimator errors), so a detector can learn the crash itself rather than its cause, a form of crash-window data leakage [8, 9]. Second, comparing crash flights with other operators' good flights mixes the flight with the aircraft and the people using it, so a difference between groups need not say anything about the flight that crashed (between-aircraft confounding). Public fault datasets are dominated by simulation or injected faults [10, 11, 12], and accident statistics classify outcomes such as loss of control rather than causes visible in a log [13, 14]; neither resource tests whether real crash logs, evaluated under leakage-free and within-aircraft protocols, can support the attribution and prioritization decisions that operators actually need.

The central question this paper asks of the largest public archive of real

autopilot logs, PX4 Flight Review [15, 16], is: *what evidentiary value can flight logs carry for a drone fleet operator, once evaluation protocols guard against crash-window leakage and between-aircraft confounding?* We decompose it into four sub-questions:

- Q1 Do configuration settings differ between crash flights and normal flights, and do they indicate who was responsible?
- Q2 Can a log separate crashes caused by the pilot from crashes caused by the aircraft?
- Q3 Which comparison, within the same aircraft or between aircraft, retains a usable signal before a crash?
- Q4 Do expert diagnoses agree with uploader labels, and do automatic checks find the faults experts name without using the crash itself?

Answering them exposed a further problem that we had not planned to test: the archive's own records (crash moments, labels and flight dates) are less reliable than such studies usually assume.

The paper makes three claims, each tested several ways, and offers practical outputs.

Claim 1: evaluation design creates much of the apparent skill. Across 32 evaluation settings the apparent crash-detection AUC fell by 0.21 [0.06, 0.34], of which 0.06 came from grouping folds by vehicle, and comparing each drone with its own earlier flights removed what remained (Sections 5.3–5.5, 5.6). The same holds outside our pipeline: on 488 real flights with injected faults, flight-style features separated faulty from fault-free flights before any fault existed, and a re-implementation of a published trained fault detector, which did not reproduce the published accuracy, fell from AUC 0.94 to 0.63 once whole days were held out (Section 5.7).

Claim 2: public crash archives need auditing before their records are used as labels.† The tilt rule that usually fixes the crash moment finds no crash-like tilt in 61% of crash-rated multicopter logs and misses roughly 7–50% of crashes, depending on the rater; pilot-rated logs show a tilt event half as often as aircraft-rated ones; a blind independent relabelling agreed with the uploaders no better than chance ($\kappa = -0.10$); and dates cannot order flights within a day (Sections 5.2, 5.4, 5.6).

Claim 3: logs support evidence review, not verdicts. No declared rule, model, published checker or expert-diagnosed comparison separated pilot-caused from aircraft-caused crashes (rule 0.47 balanced accuracy on 112 held-out crashes; models near chance, also on multicopters), and with either label

set label noise cannot be excluded (Section 5.2). Crash flights showed no warning against the drone's own earlier flights. The one positive signal is between aircraft and weak: drones that later crashed vibrated more than matched drones (airframe-clustered $p = 0.012$; site-weighted $p = 0.076$), but a vibration ranking was not shown to beat random inspection (Section 5.6).

Outputs. A ten-item reporting checklist for log-based diagnostics (Table 13), a small set of real crash logs with independent expert diagnoses (Section 5.8), and an evidence-first design for fleet-assurance tools (Section 6). Code, declarations, derived features and labels are released; analyses added after the declared protocol are marked † and listed in Supplement S12.

2. Related work

Fault datasets. Table 1 compares our data with the main public resources. ALFA provides 47 fixed-wing flights with injected engine and control-surface failures [10]. RflyMAD covers 5,629 multicopter fault cases with eleven fault types in simulation and seven in real flights, of which 497 are real flights and the rest software- or hardware-in-the-loop simulations [11]. UAV-SEAD offers about 1,396 real PX4 logs annotated for state-estimation anomalies [12], used by later anomaly-mining work [17]; a recent testbed quantifies the discrepancies between simulated and real PX4 logs [18]. These resources target anomaly or fault detection under known faults. Wang et al. collected 178 real abnormal PX4 and ArduPilot logs from software-bug incidents and studied those that led to crashes [19]; their labels are software root causes. To our knowledge, none of these resources labels real crashes by pilot-versus-aircraft responsibility or separates the pre-crash window from the crash.

Fault detection and diagnosis. Surveys of fault detection for UAVs and robots cover model-based and data-driven methods and stress how hard robotic diagnosis is in the field [20, 21]. Causal models built from autopilot documentation have been used to find actual causes of failures in ArduPilot logs [22]. A study of 569 bugs in PX4 and ArduPilot found parameter handling among the root-cause types of drone-specific bugs [23], and a later study collected 178 abnormal PX4 and ArduPilot flight logs from real software-bug incidents [19]. MAYDAY argues that flight logs record only high-level control states, so an accident whose cause lies in the control program often cannot be traced from the log alone; it adds program-level logging and traced 10 ArduPilot accidents to control-semantic bugs [24]. Our negative attribution result is the

Table 1: Public UAV fault and anomaly data compared with this study.

Resource	Flights	Faults or labels	Real crashes labelled by cause (to our knowledge)
ALFA [10]	47 fixed-wing flights	Injected engine and control-surface failures	No
RflyMAD [11]	5,629 cases (497 real)	11 injected fault types (7 in real flights)	No
UAV-SEAD [12]	about 1,396 real PX4 logs	State-estimation anomalies (4 classes)	No
Wang et al. [19]	178 real abnormal PX4/ArduPilot logs	Software-bug root causes	Partly (software bugs only; not responsibility)
This study	479,547-log index; 590 crash-rated logs; 148 expert-diagnosed threads	Uploader responsibility labels; expert causes	Yes (uploader; partly expert-checked; to our knowledge)

statistical counterpart of that argument for pilot-versus-aircraft calls. Recent work recovers and parses damaged ArduPilot, PX4 and Betaflight logs for forensic use [25], and diagnoses battery-induced crashes from hexacopter flight logs in controlled scenarios [26]; both study logs under known conditions rather than an uncurated archive. Gryphon extracts critical events from ArduPilot logs with a six-step forensic method that ends in a timeline [27]; AeroFleet's evidence timeline follows the same principle and adds no attribution. Maintenance-oriented reliability analysis of UAVs models the passage from partial degradation to failure [28], the view our between-drone result supports.

Accident analysis. Analyses of civil drone occurrences find equipment problems and loss of control prominent [13]. Human-factors frameworks for remotely piloted aircraft (such as the Human Factors Analysis and Classification System, HFACS) model mishaps as multi-causal sequences involving organizational influences, environmental preconditions, and supervisory fail-

ures alongside direct operator actions [7, 29]. In operational practice, post-incident reporting and warranty claims often collapse these complex systemic interactions into a single nominal party: pilot versus aircraft. Because such classifications depend heavily on the rater and framing, we evaluate agreement against expert community diagnoses.

Language models for diagnosis. Language-model agents have been applied to simulation testing and log analytics for small UAVs [5] and to UAV forensic investigation [30], and learned models to classifying problems in drone log messages for forensics [6]. Multi-agent debate has been proposed to improve the factuality of model outputs [31], while indirect prompt injection shows that text inside data can steer model behaviour [32]. We test both properties on our task.

Methodology. Leakage, where information unavailable at prediction time enters training or evaluation, is a known source of irreproducible machine-learning results [8, 9]; in bearing fault diagnosis, splitting by segment rather than by physical bearing inflates reported accuracy, and bearing-wise partitioning is proposed as the fix [33]. Grouping folds by unit is therefore not new; what we add is a measurement of how much it, and four other evaluation choices, change results on real crash logs; label noise likewise biases evaluation [34]. In the drone-log domain we identify a domain-specific variant: *crash-window leakage*, in which the physical consequences of the crash (saturated actuators, estimator divergence, attitude extremes) contaminate features used to evaluate pre-crash diagnostics. Matching reduces confounding in observational comparisons [35], and declaring hypotheses and analysis scripts in version control prior to running tests separates confirmatory from exploratory analysis [36]. We apply these ideas together as an evaluation protocol for drone logs.

Literature search. One author searched IEEE Xplore, the ACM Digital Library, ScienceDirect, arXiv and Google Scholar for 2016–2026 with three query strings (crash logs and attribution; leakage and cross-validation in UAV anomaly detection; vibration-based maintenance), recorded with the inclusion rules in the released literature-search protocol (26 October 2026). Four closely related studies [24, 27, 19, 33] were missed by that search and added on 27 October. Forward citation chasing on three of them (Semantic Scholar, 27 October 2026) added three more [26, 25, 30]; chasing for MAY-

DAY could not be completed through the public API. The search is not a systematic review.

3. Data

3.1. Archive and labels

PX4 Flight Review is a public service to which PX4 users upload onboard logs (ULog format) for plotting and sharing [16]. We downloaded its public index on 20 October 2026: 479,547 logs (32,793,172 bytes, SHA-256 `1bbb01c3…a92d6`), of which 442,929 came from real hardware rather than simulation, from 38,320 distinct vehicle identifiers. Uploaders may rate a log. A total of 590 logs carry an uploader crash rating (402 aircraft-caused, 188 pilot-caused); 70 of them come from simulation, and Table 2 details the 520 from real hardware. These uploader-assigned responsibility labels are not independently verified ground truth; they reflect the uploaders' own assessments. A notable discrepancy appears between uploader ratings and expert conclusions: while uploaders attributed 31.9% of rated crashes to the pilot (188 of 590), forum experts blamed the pilot in only 6.1% of all threads (9 of 148) and 8.6% of decided threads (9 of 105, Section 5.8). Across all 148 real crash threads, download attempts yielded 43 downloadable logs from 43 distinct threads (38 aircraft-fault and 5 pilot-fault; the remaining 105 had unhosted or expired external links). Cross-tabulating these 43 logs against archive ratings reveals that 9 carried uploader aircraft crash ratings (`crash_sw_hw`, with 100% expert agreement: all 9 diagnosed as aircraft faults), 33 were unrated in the archive index (28 aircraft, 5 pilot), 1 was rated unsatisfactory (aircraft), and exactly 0 were uploaded with a `crash_pilot` rating. This pattern is consistent with, but not statistical evidence of, reluctance to self-identify pilot fault: all 5 pilot logs were unrated in the index against 28 of 38 aircraft logs unrated (one-sided Fisher exact $p = 0.25$). Across the wider set of all 148 forum threads, 14 referenced logs that carried an uploader aircraft crash rating in the archive index (experts agreed with the aircraft fault in 11 and were unclear in 3; none blamed the pilot); 9 of these 14 threads were among the 43 with surviving, downloadable log files (all 9 with complete expert agreement), while 5 had expired external links. This gap may reflect selection bias in which incidents operators submit for community assistance, or a narrower definition of pilot error among autopilot developers compared to end users. Forum diagnoses represent informed community opinion rather than independently verified accident inspection

Table 2: Composition of crash-rated and well-rated logs from real hardware in the archive.

	Crash-rated	Rated good or great
Logs	520	21,112
Median duration	54 s	388 s
Most frequent years	2018 (91), 2025 (89), 2019 (87), 2017 (57)	2018 (10,571), 2017 (5,748), 2019 (2,910)
Most frequent PX4 versions	1.8 (70), 1.9 (66), 1.15 (65), 1.11 (65)	1.7 (6,299), 1.8 (5,785), 1.6 (4,265)
Vehicle types	quadrotor 320, fixed wing 71, hexarotor 38	quadrotor 19,890, fixed wing 620, hexarotor 291

ground truth; original posters explicitly confirmed the diagnosis in only 47 of 94 aircraft threads and 5 of 9 pilot threads. Logs rated "good" or "great" served as normal flights. Logs were parsed with pyulog. We store derived features and log identifiers, not the logs.

A ULog file contains the parameters in force when logging started, any parameter changes during logging, timestamped messages published by the autopilot's modules, and logged text messages with a severity level. The analyses below use the vehicle status (arming state, navigation mode, failsafe flags), attitude and body-rate topics, rate setpoints, inertial-sensor status, actuator outputs, battery status, GPS position and the land detector. The vehicle identifier in the index is derived from the flight controller's hardware serial number; it identifies a flight controller, which in practice usually means one airframe, though a controller can be moved between frames.

Crash-rated logs differ from well-rated logs in composition (Table 2): they are shorter (median 54 s against 388 s, partly because a crash ends the log) and spread over more firmware generations. All between-group comparisons below therefore use either matching or within-drone designs, and we do not pool across firmware generations where measurements differ.

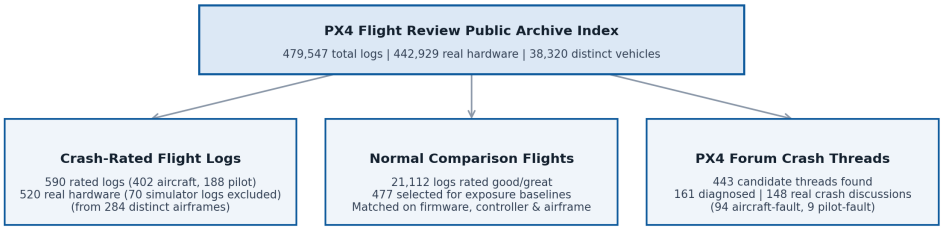
3.2. Samples

Figure 1 and Table 3 show how each analysis sample was drawn. We evaluate two primary classification tasks: Task A (separating crash flights from normal flights) and Task B (binary attribution of crash responsibility between drone and pilot). We distinguish two distinct cohorts that each happen to number 148: (1) the leak-curve and attribution modeling sample of 148 crashes and 139 normal flights at $m = 0$ s (Section 4), and (2) the

expert community sample of 148 crash discussion threads on the PX4 forum (Section 5.8). The attribution rule was evaluated on 205 crashes (93 development and 112 held-out). All 148 crashes entering the statistical attribution models are a strict complete-case subset of these 205 crashes (100% overlap); 57 crashes were excluded from modeling because required ULog topics were unrecorded or the flight window before cut-off was shorter than 10 s. In the leak-curve and attribution modeling sample, sample sizes across cut-off margins $m \in \{0, 2, 5, 10, 20, 30\}$ s reflect progressive attrition as flights with duration under 10 s before cut-off are dropped: for Task A (crash vs. normal), 148 vs. 139 ($m = 0$ s), 142 vs. 137 ($m = 2$ s), 135 vs. 133 ($m = 5$ s), 114 vs. 130 ($m = 10$ s), 100 vs. 128 ($m = 20$ s), and 89 vs. 124 ($m = 30$ s); for Task B (drone vs. pilot responsibility among crashes), 112 vs. 36 ($m = 0$ s, $n = 148$), 106 vs. 36 ($m = 2$ s, $n = 142$), 101 vs. 34 ($m = 5$ s, $n = 135$), 87 vs. 27 ($m = 10$ s, $n = 114$), 73 vs. 27 ($m = 20$ s, $n = 100$), and 63 vs. 26 ($m = 30$ s, $n = 89$). A constant complete-case subset of 213 flights (89 crashes and 124 normal flights; 63 drone and 26 pilot crashes) with pre-crash duration ≥ 40 s was evaluated across all six margins without attrition.

Initial cross-sectional samples for Q1 and Q2 selected flights with PX4 1.10 or newer and at least 5 s of flight; normal flights were matched to crashes on vehicle type and drawn from vehicles without any crash rating. For the longitudinal within-drone and between-drone tests (Q3b, Q3c), only the eligible case count ($n = 12$), not any outcome or feature data, was inspected before deciding to lift the firmware restriction to avoid sample starvation; flights were instead matched within each drone's firmware generation and measurement source. For the within-drone test every crash with a vehicle identifier and at least three earlier non-crash flights of 30 s or more was eligible (101 crashes; up to the five most recent earlier flights each). For the between-drone test each crashed drone was matched with up to two drones without any crash-rated log that shared the flight-controller hardware, the PX4 major.minor version and the vehicle type (66 matched sets, 124 control slots drawn from 107 unique non-crash control drones, three randomly chosen flights each, seed 7). Control drones were sampled without replacement within each matched set. The three exposure flights of a crashed drone were its most recent non-crash flights dated on or before the crash; flights on the crash day were included in the declared test because the archive does not give their order within a day, and removed in a sensitivity analysis.

Not all selected flights enter every test. In the within-drone candidate pool (101 crashes with earlier flights), 34 crashed drones were dropped be-



Analysis Stream	Initial Candidate Pool	Exclusions & Attrition	Analyzed Sample Size
Q1: Parameters	100 crash, 477 normal	43% crash logs lack factory defaults	57 crash, 113 normal
Q2: Fixed Rule	205 crashes, 143 normal	None (evaluates 93 dev + 112 held-out)	205 crashes, 143 normal
Q2/Q3a: Leak Curve	205 crashes, 143 normal	57 dropped (missing topics / window < 10 s)	148 crash, 139 normal (m=0 s) 89 crash, 124 normal (m=30 s)
Q3b: Within-Drone	101 crashed drones	34 dropped (single flight / window < 10 s)	67 multi-flight drones (372 fl.) 61 vibration, 46 tracking
Q3c: Between-Drone	66 matched sets (101 cases)	6 sets dropped (unmatched sensor metric)	60 vibration sets (124 ctrl, 372 fl.) 47 rate-tracking sets
Q4: Expert Cases	148 real crash threads	105 logs unhosted or expired links	43 downloadable (38 aircraft, 5 pilot) 35 aircraft with fault categories

Figure 1: From the public index to each analysis. Counts are logs unless stated.

cause they lacked multiple valid flights or had duration under 10 s before cut-off, leaving 67 multi-flight drones (372 total flights: 67 crash and 305 earlier flights). Among these 372 flights, 55 individual flight logs could not be read or had under 10 s before cut-off; requiring that crash and earlier flights share the same measurement source left 61 analysed drones for vibration, and restricting to rotary-wing vehicles with rate setpoints left 46 analysed drones for rate tracking. In the between-drone sample (66 matched sets), 71 flights were unreadable or too short; a drone's value requires at least two readable flights, and case and controls must share the measurement source, leaving 60 analysed sets for vibration and 47 for tracking. In the between-drone sample, 124 matched control slots contributed 372 flights (124×3), distinct from the 372 flights (67 crash flights and 305 earlier flights) evaluated in the within-drone longitudinal model.

3.3. Expert diagnoses

We searched the PX4 discussion forum for threads that link a Flight Review log and mention a crash (443 threads, 410 with a log link). For the

Table 3: Analysis samples and evaluation cohorts.

Sample	Crashes	Comparison	Question
Settings	100 (57 with default records)	477 normal (113 with default records)	Q1
Development crashes	93 analysed	–	Q2 rule
Held-out crashes	112 analysed	143 normal (full logs)	Q2 replication
Leak-curve set	148 at $m = 0\,\mathrm{s}$ (89 at $m = 30\,\mathrm{s}$)	139 normal (124 at $m = 30\,\mathrm{s}$)	Q2 model, Q3a
Constant-subset leak curve	89 crashes ($\geq 40\,\mathrm{s}$ duration)	124 normal ($n = 213$ constant across margins)	Q2 model, Q3a
Within-drone rank test	101 selected (61 and 46 analysed)	475 earlier flights of the same drones	Q3b
Within-drone predictive model	67 multi-flight drones (67 pre-crash)	305 earlier flights of same drones ($n = 372$)	Q3b
Between-drone matched sets	66 matched sets (60 and 47 analysed)	124 matched drones, 372 flights	Q3c
Expert forum cases	148 real crash threads	43 logs downloadable (38 aircraft, 5 pilot)	Q4

161 threads with a reply from PX4 staff, the development team or a member at the forum's highest trust levels (148 of them describing a real crash), we recorded the expert's conclusion (aircraft, pilot, both, unclear, no diagnosis), the cause named, who gave it, whether the original poster confirmed it, and the sentence stating the diagnosis. Recording was AI-assisted and reviewed by an author; 123 of 124 recorded sentences were found verbatim in their

thread by script. A second author then coded all threads blind to the first recording, with the same categories; a script confirmed that 150 of 151 coded rows quoted their own thread or recorded no diagnosis. On the 147 threads both coded, agreement (Cohen's kappa, 95% bootstrap interval) was 0.81 (0.71–0.88) for the expert's conclusion, 0.73 (0.65–0.81) for the cause and 0.79 (0.69–0.88) for whether the poster confirmed it, above the declared pass mark of 0.60. The two coders settled the 45 threads that differed on conclusion or cause by discussion, which changed 13 labels; counts and tests below use the settled labels, and the first recording led to the same conclusions (both are in the repository). The searches combined the Flight Review host names with the words crash, crashed, flyaway, fell, flipped and lost control, and every returned thread was retrieved through the forum's public interface. A reply counted as expert when its author was marked as staff, belonged to the development team group or held one of the forum's two highest trust levels. Only public posts were used; usernames and post text are not redistributed.

4. Methods

4.1. Declared test protocols

The public archive index was downloaded on 20 October 2026. For each confirmatory test, the hypothesis, sample selection, features, statistic, and pass mark were declared and committed to version control before the analysis it governs was run (Table 4). Where a test reused logs already downloaded for an earlier test, the protocol records it as declared before running, not before download (exploratory analyses added after the declared protocol are marked with †). Unlike third-party timestamped registries (such as the Open Science Framework), commit timestamps in self-hosted git remain author-controlled until an independent snapshot is indexed; the git commit history establishes an immutable cryptographic SHA-1 chain in version control, and archived snapshots have been deposited in Software Heritage (`swh:1:rev:bad06ff1256865c7e63d44389aac174e0281291d`, `swh:1:rev:bbdd0420de4547ac7d67b84165292a46237b604d`, `swh:1:rev:624dd694e0d207dd9c44b1c9550faaf04453bc56`, and `swh:1:rev:3a9aa89417fa9ea2a2334e3a4794e2358fb713ee`) alongside the repository's git commit hashes (including `eb463b5` for the decomposition), and will be minted into a Zenodo archive DOI upon publication. One amendment was data-dependent: after observing that the firmware filter of the within-drone test left only 12 eligible cases under the initial ≥ 1.10 filter (without inspecting any outcome data), the firmware

Table 4: Declaration and results commits for each test (IST, from the git history; author-controlled until archived). Each test was run between its declaration commit and the commit that first added its result file, so the run time is bounded by those two times. Tests sharing a script share its declaration (for example, the clipping and manual-mode sign tests are in the Q2 rule script; within- and between-drone tracking are in the Q3b and Q3c scripts).

Test	Focus (script)	Declared	Results committed	Status
Q1	Protections disabled, Fisher exact (`analyze.py`)	`d755cca` 20 Oct 17:23:31 (code and result together)		First pass, not declared before running
Q2 dev	Attribution rule, clipping and manual-mode signs (`pilot_vs_drone.py`)	`63bc72e` 20 Oct 17:25:49	`19a9318` 17:26:57	Confirmatory
Q2 held	Held-out rule replication (`holdout_test.py`)	`af6fab2` 20 Oct 17:28:42	`da92ec3` 17:40:02	Confirmatory
Q4, $m = 0$	Expert fault checks (`test_flight_check.py`)	`4475c8c` 20 Oct 21:39:26	`da5038b` 21:46:47	Confirmatory
Q4, $m = 10$ s	Same checks, last 10 s removed	`0c8ee47` 20 Oct 21:46:54	`95d873a` 21:48:02	Confirmatory
Q3a	Leak curve, $\Delta \geq 0.10$ (`leak_curve.py`)	`d5d56c7` 23 Oct 11:17:11	`1ecb4c9` 11:19:07	Confirmatory
Q3b	Within-drone vibration and tracking (`within_drone.py`)	`8b417d2` 23 Oct 11:21:19; amended `f911bcf` 11:22:21	`c9ebca4` 12:19:44	Confirmatory (amended before download)
Q3c	Between-drone vibration and tracking (`rough_drone_test.py`)	`d7a6574` 23 Oct 12:30:19	`a76babb` 18:15:54	Confirmatory
Replication	Newer-metric vibration (`rough_drone_newer.py`)	`2148644` 24 Oct 00:00:48	`b25d596` 00:40:24	Confirmatory
Coding	Second coder, $\kappa \geq 0.60$ (`score_forum.py`)	`21af373` 24 Oct 00:03:29	`a369f83` 12:16:14	Confirmatory

filter was removed and a fallback for older firmware was added. One coding error was corrected after a first run (logs whose CPU-load record lacked the load field were being skipped); no threshold changed. Secondary sensitivity analyses and diagnostic evaluations added after the declared protocol are explicitly designated as exploratory and documented with commit hashes in S12.

4.2. Crash moment and analysis window

The crash moment t_c is the first time after take-off at which the tilt $\theta = \arccos\left(1 - 2(q_x^2 + q_y^2)\right)$, computed from the attitude quaternion, exceeds 60°; when no such tilt occurs, it is the end of the armed period. The tilt rule is skipped only when the log's vehicle-status record marks the aircraft as not rotary-wing. Older logs lack that flag and are treated as rotary-wing, so

the rule was also applied to some fixed-wing and VTOL logs, where a bank above 60° can be normal flight. Aircraft type is taken from the archive index: multicopter means quadrotor, hexarotor or octorotor; fixed-wing means fixed wing; VTOL means any index type marked as vertical take-off and landing (tailsitters, tiltrotors and fixed-wing aircraft with lift rotors), which hover like multicopters but fly forward on a wing. Crash-moment analyses are therefore reported for multicopters (Section 5.4). Normal flights use the end of the armed period. Take-off is the first sample after arming at which the land detector reports the vehicle airborne. An analysis with cut-off m uses the window from take-off to $t_c - m$ and is skipped if the window is shorter than 10 s. Because normal flights include the complete descent, landing and disarm sequence while crash windows terminate at impact or tilt, this introduces an inherent windowing asymmetry that contributes to differences in duration and whole-window aggregates.

4.3. Features

Table 5 defines the features. Vibration is not measured the same way across firmware generations: newer logs report an accelerometer vibration metric, older ones an estimator vibration estimate. Comparisons are therefore only made between flights measured with the same source, and the source is recorded with every value. The rate-tracking error falls back to the attitude message's body rates when the angular-velocity topic is absent.

4.4. Tests

Q1 settings. For logs that carry the firmware's record of default values, a protection was counted as disabled when its value differed from the default and switched it off (for example a failsafe action of "none"). Crash and normal logs were compared with Fisher's exact test.

Q2 attribution. Task B is defined as the binary attribution of crashes between pilot and aircraft fault (contrasted with Task A, which evaluates separating crash flights from normal flights). The declared rule labelled a crash "aircraft" if any of eight signs fired before t_c (a critical battery warning or remaining charge below 10%; any motor at 95% or more for at least 1 s in the last 10 s; more than 50 clipping events; poor GPS in a GPS-dependent mode; a failure-detector flag; an error-level message; CPU load above 95%; an active failsafe), otherwise "pilot" if the mode at t_c was pilot-flown, otherwise "unknown". The pass mark was a balanced accuracy of at least 0.70

Table 5: Features computed on each analysis window.

Feature	Definition
Rate-tracking error	RMS of commanded minus measured body rate, roll and pitch ($\mathrm{rad\,s^{-1}}$; rotary wing)
Tilt (p95)	95th percentile of tilt angle (deg)
Vibration (p95)	95th percentile of the accelerometer vibration metric, or of the estimator vibration estimate on older firmware
Clipping rate	accelerometer clipping events per second
Motor saturation	share of time with any motor output at 95% or more (rotary wing)
Battery minimum	lowest reported remaining charge (%)
Voltage sag	(start voltage − minimum voltage) / start voltage
Poor GPS	share of time with horizontal accuracy above 5 m or fewer than 6 satellites
Failsafe	1 if a failsafe was active at any time
Manual share	share of time in a pilot-flown mode (manual, altitude, position, acro, stabilised)
Stick deflection (p95)	95th percentile of the larger roll or pitch stick deflection

on decided cases, answering at least 60% of crashes and being wrong on at most 20%. A model-based check used L2-regularised logistic regression on all features ($\lambda = 1$, standardised inputs, median imputation within training folds), evaluated by stratified five-fold cross-validation repeated five times and summarised by the ROC AUC [37]. As an exploratory check on non-linear threshold effects, single-split decision stumps were evaluated alongside logistic regression. We also evaluated depth-3 decision trees across all cut-offs ($m \in \{0, 2, 5, 10, 20, 30\}$ s). Logistic models, stumps and trees are exploratory; their uncertainty is assessed against a permutation null that refits the whole cross-validation (Section 5.2).

The signs were chosen before the analysis to cover common failure modes of PX4 multirotors: power, propulsion, vibration, navigation, software and failsafes. The pilot rule was deliberately simple: if no aircraft sign fired and the vehicle was in a mode in which the pilot controls attitude or position directly, the crash was attributed to the pilot. The rule was fixed before it was run on any crash log; only the list of message types available in the logs had been inspected.

Q3a crash window. The same model separated crash from normal flights at cut-offs $m \in \{0, 2, 5, 10, 20, 30\}$ s. We declared that the AUC would fall by at least 0.10 between $m = 0$ and $m = 10$ s if crash consequences drove it.

Q3b and Q3c rank tests. For a crashed drone i with value x_{i0} and comparison values $x_{i1}, \ldots, x_{in_i}$, the scaled rank is

$$r_i = \frac{1}{n_i} \sum_{j=1}^{n_i} \mathbf{1}\left[x_{ij} < x_{i0}\right], \tag{1}$$

which is 0 when the crashed drone has the lowest value and 1 when it has the highest; ties count against the hypothesis. A rank statistic was chosen for three reasons: the two vibration sources have different units and scales, so only orderings are comparable across drones; the number of comparisons differs between drones, which the exact null handles directly; and the distributions are skewed, with a few very rough flights, which would dominate a mean difference. In the within-drone test (Q3b) x_{i0} is the crash flight, cut 10 s before t_c, and the comparisons are the same drone's earlier flights, each cut 10 s before the end of its armed period. In the between-drone test (Q3c) x_{i0} is the median over the crashed drone's three most recent pre-crash flights and the comparisons are the medians of the matched controls. Under no difference each r_i is uniform on $\{0, 1/n_i, \ldots, 1\}$ with mean 0.5, and $\bar{r}$ estimates the probability that the crashed drone exceeds a comparison, the quantity estimated by the Mann–Whitney statistic [38]. One-sided p-values come from 100,000 draws of this exact null; 95% confidence intervals from 10,000 bootstrap resamples of drones [39]. Null rates for the share of sets in which the crashed drone is the roughest, and a duration adjustment used as a sensitivity analysis, are given in Supplement S5. Each test had two primary features (vibration, rate-tracking error) at $\alpha = 0.025$.

Q4 expert diagnoses. Expert conclusions were compared with uploader labels. Fixed checks for the five physical failure modes experts named (tuning, vibration, GPS or compass, motor or ESC, battery; definitions in Supplement S7) were declared useful if they fired on at least 50% of logs with that diagnosis and at most 10% of normal flights, first on the full window and then with the last 10 s removed.

Multiplicity. Seven tests produce p-values. Holm's procedure [40] is reported over all seven in addition to each test's declared α. The Q1 settings comparison came from a first-pass analysis whose code and result were committed

together (Table 4), so it is not confirmatory; it is kept in the Holm set, which makes the adjustment of the other tests conservative.

5. Results

Figure 2 shows the two main results at a glance, and Table 6 summarises every finding and how strongly it is supported; the subsections give the detail.

Table 6: Main findings at a glance. "Declared": hypothesis and pass mark committed before the test ran; "exploratory": added after the declared protocol († in the text).

Question	Finding	Evidence	Verdict	Section
Pilot or aircraft? (Q2)	Rule 0.47 balanced accuracy on 112 held-out crashes (0.43 on 86 multicopters); models AUC 0.42–0.59 (multicopters 0.50–0.64, none after Holm)	Declared rule; exploratory models	No usable signal	5.2
Settings (Q1)	Protections disabled in 51% of crash and 46% of normal logs ($p = 0.63$)	First pass, descriptive	No difference	5.1
Expert agreement (Q4)	Rule vs. 43 expert-diagnosed logs: balanced accuracy 0.38 (CrI 0.25–0.61)	Exploratory	Inconclusive	5.8
Crash window (Q3a)	AUC drop 0.035 when the last 10 s are cut (criterion 0.10); window effect 0.089 [0.019, 0.168] on tilt-defined multicopter crashes ($n = 37$), 0.131 without tilt and stick features	Declared; post hoc subset	Small overall, material on tilt-defined multicopters	5.3, 5.4
Warning before a crash (Q3b)	Crash flight like the same drone's earlier flights (vibration rank 0.48, $p = 0.67$)	Declared	No warning	5.6
Which drones crash (Q3c)	Later-crashed drones vibrated more than matched drones (rank 0.67; airframe-clustered $p = 0.012$; site-weighted $p = 0.076$)	Declared; dependence tests exploratory	Suggestive	5.6
Screening value†	Against 412 non-crash drones, flag rates measured on held-out drones: lift 1.77× [1.00, 2.60] (leave-one-out 1.68× [0.94, 3.31])	Exploratory, declared	Not shown to beat random inspection	5.6
Evaluation choices	AUC 0.88 → 0.67 across 32 settings; 0.06 from grouping folds by vehicle	Exploratory	Fold grouping matters	5.5

Question	Finding	Evidence	Verdict	Section
Crash moment	Tilt rule finds no crash-like tilt in 61% of multicopter crash logs and misses roughly 7–50% of crashes, depending on the rater; fallback moment mostly wrong (three raters, Fleiss' $\kappa = 0.71$); two detectors failed validation	Exploratory	Tilt rule misses 7–50% of crashes	5.4
Label confound	Crash-like tilt in 23% of pilot-rated vs. 45% of aircraft-rated logs (adjusted OR 2.1 [1.2, 3.7])	Exploratory	Label tracks log content	5.4
Independent relabelling	Blind non-author reviewer vs. uploader: $\kappa = -0.10$ [−0.34, 0.17] on 58 crashes; rule vs. reviewer BA 0.55 [0.42, 0.66]	Exploratory, one reviewer	Neither label set recovered; label noise not excluded	5.2
Flight order	44–55% of dated crash-day "earlier" flights flown after the crash	Exploratory	Dates insufficient	5.6
Published log checker	FlightMD: crash vs normal AUC 0.72 [0.58, 0.81] without the last 10 s; attribution BA 0.52; no within-drone warning ($p = 0.66$)	Exploratory	Same pattern as our pipeline	5.7
Published trained detector	RflyMAD LeNet-5, re-implemented (did not reproduce the published accuracy): AUC 0.94 (random windows), 0.92 (flights held out), 0.63 (days held out); before the fault 0.85 (flights) and 0.57 (days)	Exploratory, declared	Separation tied to recording session	5.7
Injected faults (known cause)	488 real RflyMAD flights: flight-style features AUC 0.75 *before* the fault (folds grouped by day); rule, used out of domain, called 3% of motor faults aircraft-caused	Exploratory	Separation without a fault; feature-specific	5.7
Language-model panels	Worse than the fixed rule on 60 generated and 97 blind real cases; on real cases the rule itself was wrong in 64%; one compromised reviewer flipped most verdicts	Exploratory	Not recommended	5.9

5.1. Configuration settings (Q1)

Of 57 crash logs and 113 normal logs carrying a firmware-default record, 29 (51%) and 52 (46%) had at least one protection disabled relative to the default (Fisher $p = 0.63$). This comparison comes from a first-pass analysis whose code and result were committed together, so it is reported as a descriptive result rather than a declared test (Table 4). Pilot-caused crashes

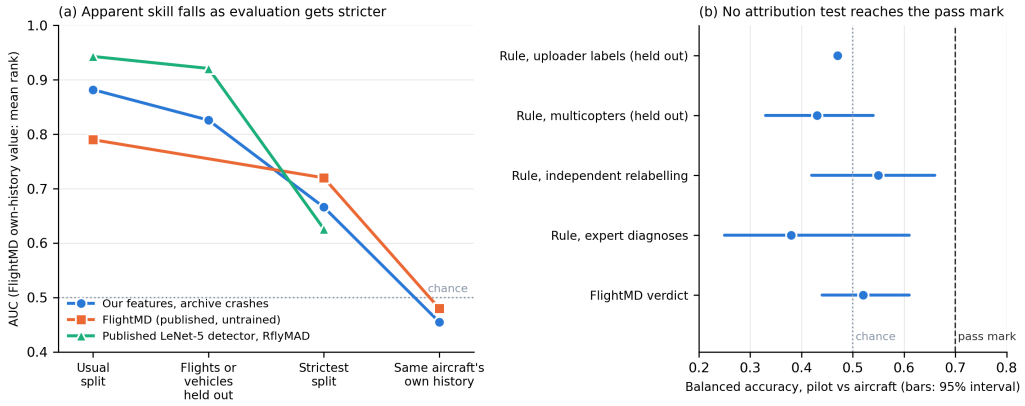


Figure 2: The paper at a glance. (a) Apparent skill falls as evaluation gets stricter. Our features: flight-stratified folds, vehicle-grouped folds, the most conservative factorial setting, and the same drones' earlier flights (Table 8, Section 5.5). FlightMD: whole log, last 10 s removed, and the crash flight's mean rank against the drone's own earlier flights (Section 5.7). Published LeNet-5 detector, re-implemented: random windows, flights held out, days held out (Section 5.7). (b) Balanced accuracy of the pilot-or-aircraft tests against chance and the declared 0.70 pass mark (Sections 5.2, 5.8).

had a protection disabled more often than aircraft-caused ones: 14 of 20 (70%) against 15 of 37 (41%), an observed difference of +29.5% (95% CI 3.9–55.0%, odds ratio 3.42, Fisher's exact two-sided $p = 0.052$). This contrast is exploratory, and does not reach statistical significance unadjusted (Fisher's exact two-sided $p = 0.052$). Default parameter records were present only on newer firmware (missing for 43% of the 100 crash logs); on the 57 available cases, treating disabled protections as a pilot-attribution classifier yields a sensitivity of 70.0%, a specificity of 59.5%, and a balanced accuracy of 0.647, below the declared 0.70 pass mark. None of 100 crash logs changed a safety-critical or flight-limit setting during the flight.

5.2. Attributing a crash to the pilot or the aircraft (Q2)

The fixed rule reached a balanced accuracy of 0.57 on 93 development crashes (35% wrong) and 0.47 on 112 held-out crashes (47% wrong), against the declared 0.70, and labelled 39% of 143 normal flights as aircraft faults (Table 7). The signs fired at similar rates whichever party the uploader blamed (Fig. 3): motor saturation in 23% of aircraft-caused and 29% of pilot-caused crashes (1% of normal flights), battery warnings in 15% and 18% (3%). Error messages and failsafes were no more frequent in crashes

than in normal flights. Two signals noticed after the development run did not replicate on held-out crashes: accelerometer clipping (24% against 8%, $p = 0.045$) and manual mode at the crash (18% against 10%, $p = 0.165$).

†*Independent relabelling.* To see whether the uploader labels are the problem, one engineering student who is not an author relabelled 72 modelled crashes from plots alone (all 36 pilot-rated and 36 randomly drawn aircraft-rated crashes; no log identifier, date, notes, rating or Flight Review link; scoring declared in `fa5fb0b` before the pack was built; the same person was the third crash-moment rater). The reviewer chose among pilot, aircraft, environment and "can't tell" and named a main cause; they called 30 crashes aircraft-caused, 28 pilot-caused, 13 environmental and 1 undecidable. On the 58 called pilot or aircraft, agreement with the uploader was no better than chance (45%, $\kappa = -0.10$, -0.34 to 0.17). Against the reviewer's labels the declared rule reached a balanced accuracy of 0.55 [0.42, 0.66], FlightMD 0.52 [0.41, 0.64] and "tilt event present" an AUC of 0.54 [0.41, 0.66]. The two label sets therefore disagree with each other, and the tools recover neither. Because the reviewer's labels come from plots and their reliability could not be measured, this does not show that the uploader labels are adequate: label noise cannot be excluded as part of the reason for the failure.

Cross-validated logistic regression and depth-3 trees on the 148 modelled crashes gave AUC 0.42–0.59 across cut-offs. Against a permutation null that refits the whole cross-validation, no cell had $p < 0.05$, and approximate upper bounds were at most 0.69. With 36 pilot-labelled crashes only an AUC of about 0.72 could be detected with 80% power, while an operational verdict would need about 0.8, which would take roughly 100–200 independently verified pilot-fault crashes. Only the rule was declared; the models are exploratory (details, the rule across cut-offs and the sample-size calculation in Supplement S1). †Because tilt features are noise for fixed-wing and VTOL aircraft, we repeated the analysis on the 108 modelled multicopters (27 pilot-rated; declared `e45b583`). The rule reached a balanced accuracy of 0.43 [0.33, 0.54] on the 86 held-out multicopter crashes and 0.37 [0.28, 0.47] on the 108 modelled ones. Logistic regression gave AUC 0.52–0.58 at every cut-off (no $p < 0.05$). Depth-3 trees gave 0.61–0.64 while the last 0–5 s were kept (unadjusted $p \approx 0.02$; Holm-adjusted over these 12 cells 0.18–0.22, and about 0.54 for the smallest, $p = 0.015$, if the 24 earlier Task B cells are included) and 0.50–0.55 once the last 10 s or more were removed, so their modest separation depends on the crash's final seconds rather than on anything before it. With 27 pilot-rated multicopters only an AUC of about 0.72 could be detected

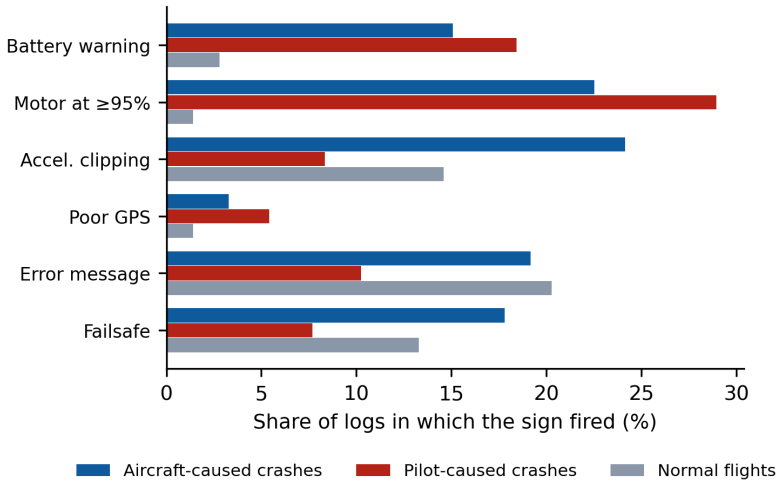


Figure 3: How often each aircraft-fault sign fired in held-out crashes, by uploader label, and in normal flights.

with 80% power. Counting undecided crashes as wrong lowers these values, but not by much: on decided crashes alone the rule reached 0.46 [0.35, 0.56] held out and 0.40 [0.31, 0.51] on the 108 modelled multicopters (declared check `1e0bf0f`), because an aircraft sign fired on most pilot-rated crashes (pilot recall 19–34%); this is no usable signal, and if anything the association is reversed, plausibly because impacts in pilot-rated crashes trigger the signs (motor saturation is checked in the last 10 s before the crash moment); a rule with its verdicts flipped would reach about 0.60, still below the 0.70 pass mark. By the declared rule the result stays "near chance": no model was reliably above chance, and the trees depended on the crash's final seconds.

Table 7: Tests, their declared pass marks and outcomes. Holm adjustment over the seven p-valued tests, first with the declared between-drone p and then (†) with the site-weighted $p = 0.076$ in its place, which reorders the tests. ‡First-pass analysis, not declared before running (Table 4).

Test (declared criterion)	Result	Holm p	Outcome
Protections disabled, crash vs normal‡	51% vs 46%, $p = 0.63$	1.00	No difference
Attribution rule, dev. (BA ≥ 0.70)	BA 0.57, 35% wrong	–	Failed
Attribution rule, held-out	BA 0.47, 47% wrong	–	Failed
Clipping, aircraft vs pilot ($p < 0.025$)	24% vs 8%, $p = 0.045$	0.27; †0.32	Not supported
Manual mode, pilot vs aircraft ($p < 0.025$)	18% vs 10%, $p = 0.165$	0.73; †0.75	Not supported
AUC drop from 0 to 10 s (≥ 0.10)	0.88 to 0.87 (stratified); 0.833 to 0.799 (grouped, one cohort)	–	Not supported
Within-drone vibration ($p < 0.025$)	$\bar{r} = 0.48$, $p = 0.67$	1.00	Not supported
Within-drone tracking ($p < 0.025$)	$\bar{r} = 0.50$, $p = 0.47$	1.00	Not supported
Between-drone vibration ($p < 0.025$)	$\bar{r} = 0.67$, $p = 0.0023$; †airframe-clustered $p = 0.012$; site-weighted (primary dependence test) $p = 0.076$	0.016; †0.08; 0.45	Supported as declared; suggestive once operator sites are weighted equally; replication inconclusive
Between-drone tracking ($p < 0.025$)	$\bar{r} = 0.57$, $p = 0.15$	0.73; †0.75	Not supported
Expert-fault checks, $m =$ 10 s ($\geq 50\%$, $\leq 10\%$)	best: vibration 3/5 vs 11%	–	None met

5.3. The crash window (Q3a)

In the declared sample, crash flights separated from normal flights at AUC 0.88 with flight-stratified folds, and removing the last 10 s barely changed this (0.87), so the declared leak criterion (a drop of at least 0.10) was not met. With folds grouped by vehicle on the constant cohort of 213 flights, AUC was 0.833 at $m = 0$ s and 0.799 at $m = 10$ s, a drop of 0.035 [0.005, 0.077] (Table 8). The separation therefore does not rest mainly on the crash itself. Crash flights were shorter (median 71 s against 169 s; duration alone gives AUC 0.66), and grouping folds by vehicle lowered AUC from 0.882 to 0.826, because 185 of the 287 flights come from 34 vehicles with several flights each. Individual checks did leak (exploratory): the rate-tracking feature fell from AUC 0.81 to 0.69 as the crash was removed, and a tuning check that found 7 of 9 expert-diagnosed tuning faults with the crash included found 1 of 9 without it (Section 5.8); firing on normal flights also fell, from 13% to 5%, so shorter windows reduce exposure as well, and the differential test gave only $p = 0.057$. Single-feature AUCs, sign firing rates across cut-offs and the split by crash history are in Supplement S2.

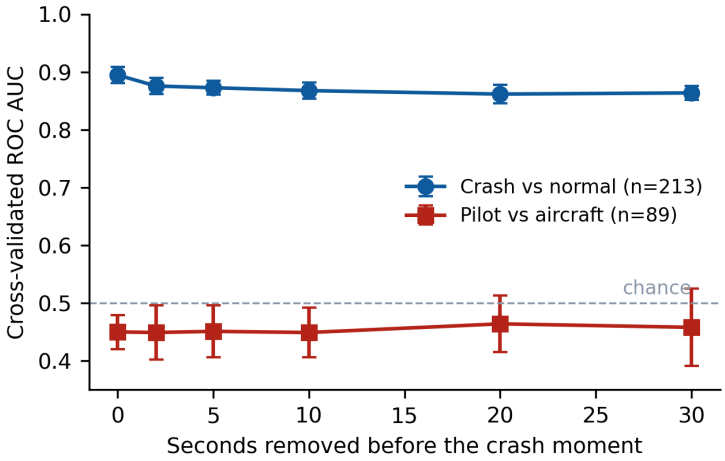


Figure 4: Cross-validated AUC on the constant subset ($n = 213$ for crash vs. normal, $n = 89$ for pilot vs. aircraft) as the end of the analysis window moves away from the crash moment. Bars: 95% bootstrap confidence intervals.

Table 8: Decomposition of model separation between crash and normal flights. Values from committed scripts with fixed seeds (`6fff27d`: seeds 42–46, clusters sorted before fold assignment, 95% cluster bootstrap over vehicles with 2,000 resamples; Step 4b from the factorial audit).

Step	Evaluation protocol specification	Sample flights	ROC AUC [95% CI]
0	Cross-sectional: stratified 5-fold CV ($m = 0$ s)	148 crash, 139 normal	0.882 [0.776, 0.935]
1	Folds grouped by vehicle	148 crash, 139 normal	0.826 [0.726, 0.895]
1b	Same, constant cohort ($m = 0$ s)†	89 crash, 124 normal	0.833 [0.730, 0.914]
2	Crash window removed ($m = 10$ s), constant cohort, grouped	89 crash, 124 normal	0.799 [0.668, 0.891]
3	Late window removed ($m = 30$ s), constant cohort, grouped	89 crash, 124 normal	0.791 [0.662, 0.889]
4	Without tilt and stick features ($m = 0$ s), grouped	148 crash, 139 normal	0.738 [0.648, 0.827]
4b	67 within-drone crashes vs. matched other-aircraft flights, one extractor ($m = 10$ s, grouped)	67 crash, 311 normal	0.623 [0.523, 0.716]
5	67 within-drone crashes vs. own earlier flights, stratified	67 crash, 305 earlier	0.546 [0.485, 0.605]
6	Same, folds grouped by drone	67 crash, 305 earlier	0.559 [0.512, 0.617]
6b	Step 6 without earlier flights flown after the crash (GPS time)	57 crash, 227 earlier	0.582 [0.525, 0.660]

The paired comparison of other-aircraft and own earlier flights uses one pipeline (recomputed features, $m = 10$ s, full window, grouped) on the same 67 crashes: Step 4b's 0.623 against 0.539 for own earlier flights (not Step 6's 0.559, which uses stored features) gives +0.083 [−0.007, 0.166]; without post-crash flights the own-flight cell is 0.510 and the difference +0.113 [0.029, 0.212]. The 0.507 quoted elsewhere is the same comparison on the factorial audit's constant subset (Section 5.5).

5.4. Crash moments and archive labels (exploratory)

Crashes with a confirmed crash moment†. The crash moment t_c is defined by tilt above 60°; we call a crash confirmed when that tilt is also followed within 10 s by disarming or a landed or ground-contact flag. Of the 148 modelled crash logs, 72 are tilt-defined (40 confirmed, 32 not) and in 76 tilt never exceeds 60°, so t_c falls back to the end of the armed period. By index vehicle type (Section 4.2) the 148 are 108 multicopters, 18 fixed-wing and 22 VTOL logs. Tilt-defined crashes are 41 of the 108 multicopters (29 confirmed), but 15 of the 18 fixed-wing (2 confirmed) and 16 of the 22 VTOL logs (9 confirmed). For fixed-wing and VTOL aircraft a bank above 60° can occur in normal flight, and an aircraft that banks and then lands and disarms meets the "confirmed" condition without crashing, so the tilt rule is not a crash criterion for them. †We therefore restrict crash-moment results to multicopters and report the mixed set only for comparison. Among the 108 multicopters, 41 (38%) are tilt-defined, the same share as in the rest of the archive (39%; see below).

†Three raters judged, by eye, the tilt and altitude traces of 50 randomly drawn crash logs: two authors, the second working blind (declared `97974ad`, `59c8212`), and a third rater who is not an author (`fdb3b7c`). They agreed well on whether the computed crash moment is right (Fleiss' $\kappa = 0.71$, 0.53–0.86): tilt-defined moments were judged right in 21, 24 and 24 of 26, including all 15 multicopter ones for the first two raters and 13 of 15 for the third, and fallback moments in only 2, 6 and 2 of 24. They disagreed on whether the fallback logs contain a crash at all (Fleiss' $\kappa = 0.33$): the first reviewer saw none in most, the others saw a crash before the computed moment that never crossed 60°. Two objective crash detectors matched the first reviewer but not the blind one and fired on 32–64% of normal flights, so both failed their declared gates and neither is used (Supplement S3).

†Because the tilt-defined crash moment on multicopters is the one the raters support (13 to 15 of 15), we repeated the window analyses on tilt-defined crashes (Table 9). These subgroups are post hoc: the tilt-defined set was chosen after the confirmed-crash result (Supplement S12), and the multicopter restriction after the review above. With grouped folds on the constant cohort (124 normal flights; intervals from a paired vehicle bootstrap, declared audit `9650a69`), removing the last 10 s lowered AUC by 0.035 [0.005, 0.077] over all 89 crashes but by 0.114 [0.027, 0.222] over the 24 tilt-defined multicopter crashes (0.878 to 0.764). That meets the declared leak criterion (≥ 0.10) by its point estimate only; the interval includes 0.10. In the factorial

Table 9: †Crash-window effect by crash subset (exploratory, post hoc). Leak drop: AUC at $m = 0$ minus AUC at $m = 10$ s, grouped folds, constant cohort with 124 normal flights. Factorial: mean $m = 0$ minus 10 s difference over the four crash and normal sets. 95% intervals from a paired vehicle bootstrap. "No tilt/stick": refitted without tilt and stick features.

Crash subset	n	Leak drop	No tilt/stick
All crashes	89	0.035 [0.005, 0.077]	0.063 [0.023, 0.109]
Tilt-defined, all types	43	0.084 [0.018, 0.139]	0.115 [0.044, 0.188]
Confirmed, all types	25	0.103 [0.027, 0.188]	0.125 [0.013, 0.247]
Tilt-defined multicopters	24	0.114 [0.027, 0.222]	0.216 [0.086, 0.402]
Confirmed multicopters	18	0.168 [0.048, 0.340]	0.155 [−0.032, 0.491]
Factorial pipeline (n is the sum over the two crash sets, e.g. 78 + 67 = 145, not unique crashes)			
All crashes	145	0.014 [−0.027, 0.038]	–
Tilt-defined, all types	63	0.067 [0.015, 0.131]	–
Tilt-defined multicopters	37	0.089 [0.019, 0.168]	0.131 [0.042, 0.226]

pipeline, the mean $m = 0$ versus 10 s difference over the four crash and normal sets was 0.014 [−0.027, 0.038] for all crashes and 0.089 [0.019, 0.168] for tilt-defined multicopter crashes (21 and 16 in the two crash sets), which meets the declared materiality rule (0.05 with an interval excluding zero).

†Because the cut removes the very tilt that defines the subgroup, part of this effect could be mechanical. We refitted without the tilt and stick features (declared before the run, with the wording rule that the abstract would drop "material" if the effect fell below 0.05). The effect grew rather than shrank: 0.216 [0.086, 0.402] on the leak curve and 0.131 [0.042, 0.226] in the factorial pipeline, because without those features AUC after the cut falls further (0.583 against 0.764 for the full model on the leak curve). The last seconds before a tilt-defined crash therefore carry signal in several features, not only the defining tilt. Crash-window leakage is small overall but material among multicopter crashes whose log shows a crash-like tilt; the small overall figure is possibly in part due to the many logs without one. The two figures of 0.014 in this paper (a Shapley share and a four-cell difference) are different quantities that coincide after rounding (Supplement S3).

Within the same drone, tilt-defined crashes still showed no warning (vibration rank 0.47, $p = 0.66$, 18 drones; tracking 0.48, $p = 0.58$, 16 drones; model 0.55 [0.43, 0.67]).

How many crash-rated logs contain the crash†*.* To see whether this holds beyond our samples, we downloaded all 520 crash-rated logs from real hardware in the archive (declared audit `09cf99c`, amended `18b0eff` to define multicopters by the index vehicle type). Of the 341 readable multicopter logs, 132 (39%) show tilt above 60° and 209 (61%) do not; our modelled sample matches this (38%). How many of these logs contain a crash depends on the reviewer: combining the archive's multicopter shares with each rater's labels on the 34 reviewed multicopter logs gives 50% [38–62%] for the first reviewer, 82% [68–93%] for the blind reviewer, 82% [68–94%] for the third rater and 67% [50–85%] for the majority of the three. Agreement on this question was low (Fleiss' $\kappa = 0.33$), so by the declared rule we report the range; on multicopters, counting fallback moments judged correct, the tilt rule therefore misses about 7% [0, 22] of crashes on the first reviewer's labels, 31% [15, 46] on the blind reviewer's, 50% [34, 65] on the third rater's and 28% [8, 46] on the majority's (declared `031fdd6`; Supplement S3). No rater's interval lies wholly above 50%, so the tilt rule misses many crashes but not demonstrably most, which is not the same as the logs lacking them. The share with a tilt event differs by label: 45% of aircraft-rated against 23% of pilot-rated logs (Fisher $p = 0.0003$; adjusted for era and vehicle type, odds ratio 2.08 [1.17, 3.69], declared audit `35103da`). The uploader's rating therefore partly encodes whether the log shows a tilt-type crash, either because recordings of pilot-caused crashes stop earlier or because those crashes involve less tilt; we cannot tell these apart. On its own, "tilt event present" separates the labels at AUC 0.61 across the archive but only 0.53 [0.43, 0.63] in the 108 modelled multicopters, and within each tilt stratum of those multicopters the attribution model gave AUC 0.63 (41 crashes, 9 pilot-rated, $p = 0.10$) and 0.62 (67 crashes, 18 pilot-rated, $p = 0.11$); combined by Fisher's method $p \approx 0.065$. With so few pilot-rated crashes these neither show nor rule out a usable signal (Supplement S3).

5.5. Factorial optimism audit (exploratory)

†The steps of Table 8 change several things at once, and their samples were built with different feature extractors. To find what the crash-vs-normal AUC depends on, we declared a factorial audit after the main analyses (commit `3c34530`; amended `f6ba424` to a float32 tolerance after its extractor check aborted on rounding differences; run 27 October 2026). Every flight was re-extracted from the raw log by one extractor, and five two-level factors were crossed: crash set (103 multi-history crashes vs. the 67 within-drone crashes),

comparison flights (uploader-rated good/great normals vs. unrated flights from drones with no crash-rated log, matched to the within-drone cases on hardware, PX4 version and vehicle type), crash-window cut ($m = 0$ vs. 10 s), fold scheme (stratified by flight vs. grouped by vehicle) and window (full flight vs. first 30 s airborne). Normal flights from any crashed vehicle were removed (39 of the 139). Intervals come from one paired two-stage cluster bootstrap (2,000 resamples over crash and control vehicles, models not refitted); a factor was declared material if its share was ≥ 0.05 with a 95% interval excluding zero. The declaration passed all abort checks (anchors 0.891 and 0.559; extractor agreement 374/374; label-permutation mean 0.495).

The declared primary 2×2 ($m = 10$ s, grouped folds, full window) did not isolate a cause (Table 10): changing the normal flights lowered AUC by 0.094 [−0.014, 0.197] and changing the crash set by 0.108 [−0.037, 0.224]. Following the pre-declared wording, the AUC depends on the comparison set; the cause is not isolated. On the constant subset (63 and 60 crashes), AUC fell from 0.878 at the most optimistic setting to 0.666 at the most conservative, a drop of 0.212 [0.062, 0.335]. An exact Shapley split assigned 0.062 [0.034, 0.087] of it to vehicle-grouped folds, the only material factor; the effect was concentrated where comparison drones contribute several flights (interaction with comparison set +0.082 [0.006, 0.122]), consistent with Step 1 of Table 8, where grouping by vehicle lowered the AUC from 0.882 to 0.826. The crash-window cut carried a small but non-zero Shapley share of 0.014 [0.007, 0.023] (on all crashes; Section 5.4 gives the larger effect on tilt-defined crashes), and the fixed 30 s window carried 0.001. Separation against other aircraft persisted at the most conservative setting (0.666 [0.557, 0.770]); replacing other-aircraft flights by the same drones' earlier flights at the same settings (first 30 s window, $m = 10$ s, grouped folds) lowered AUC by a further 0.21, from 0.666 to 0.455. †On all 67 within-drone crashes with the full window, the paired difference between other-aircraft and own earlier flights is +0.083 [−0.007, 0.166] (0.623 against 0.539), and +0.113 [0.029, 0.212] once earlier flights flown after the crash are removed (0.623 against 0.510, both from the factorial pipeline's same-aircraft cell; clustering and refit audit). A fixed 30 s window also did not reproduce the caliper lift within drones (Group-CV 0.455 [0.377, 0.507]); because that window also removes proximity to the crash, it is not a clean test of the masking explanation. One same-drone cell with flight-stratified folds fell below chance (0.393), which we attribute to splitting one drone's flights across folds and do not interpret.

Table 10: †Factorial optimism audit. Top: declared primary 2×2 (103 multi-history and 67 within-drone crashes) ($m = 10$ s, vehicle-grouped folds, full window). Bottom: exact Shapley split of the 0.212 [0.062, 0.335] AUC drop from the most optimistic to the most conservative other-aircraft setting (constant subset, 63/60 crashes). 95% intervals from a paired two-stage cluster bootstrap.

Crashes vs. comparison flights	Crashes	Comparison	AUC [95% CI]
103 vs. rated normals	78	91	0.825 [0.699, 0.908]
103 vs. matched unrated	78	311	0.686 [0.591, 0.777]
67 vs. rated normals	67	91	0.673 [0.540, 0.838]
67 vs. matched unrated	67	311	0.623 [0.523, 0.716]
Factor (optimistic → conservative)	**Shapley share of drop**		**Material**
Folds: stratified → by vehicle	0.062 [0.034, 0.087]		Yes
Comparison: rated → unrated	0.070 [−0.033, 0.157]		No
Crash set: 103 → 67	0.064 [−0.062, 0.186]		No
Crash window: $m = 0 \rightarrow 10$ s	0.014 [0.007, 0.023]		No (< 0.05)
Window: full → first 30 s	0.001 [−0.031, 0.034]		No

5.6. Within the same drone and between drones (Q3b, Q3c)

Models trained to separate each drone's crash flight from its own earlier flights gave AUC 0.546–0.582 depending on the folds and on whether flights flown after the crash were removed, and 0.507 on recomputed features, so any within-drone model signal is weak and depends on the pipeline (Table 8). Matching flight durations gave no robust gain once post-crash flights were removed (Supplement S4).

Relative to the same drone's earlier flights, the crash flight was neither more vibrating ($\bar{r} = 0.48$, 95% CI 0.38–0.58, $p = 0.67$, 61 drones) nor worse at tracking its commands ($\bar{r} = 0.50$, 0.38–0.63, $p = 0.47$, 46 drones; Fig. 5). The upper 95% bound of 0.58 rules out a within-drone rise as large as the difference seen between drones ($\bar{r} = 0.67$). Relative to matched drones without a recorded crash, drones that later crashed vibrated more on their ordinary pre-crash flights ($\bar{r} = 0.67$, 0.57–0.77, $p = 0.0023$, 60 sets; Holm-adjusted $p = 0.016$), while the tracking error did not differ significantly ($\bar{r} = 0.57$, 0.44–0.70, $p = 0.15$, 47 sets). †Those 60 sets are not independent: they come from 40 crashed airframes, 17 controls are reused, and the 40 crashed airframes come from only 23 operator sites (a location proxy: the first GPS fix, with airframes within 2 km grouped). Resampling airframes gives $p = 0.012$

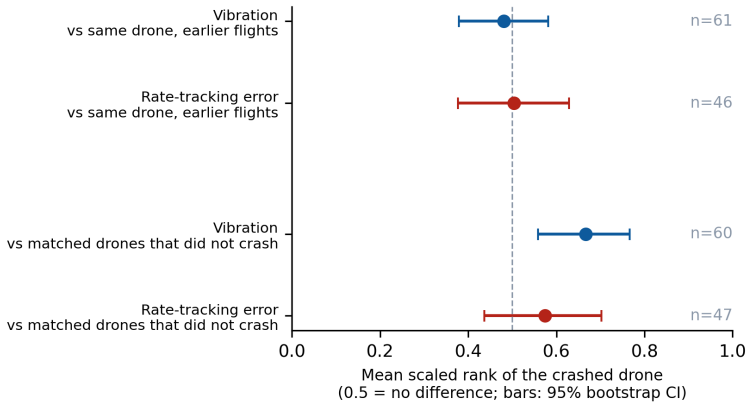


Figure 5: Mean scaled rank of the crashed drone against its own earlier flights (top) and against matched drones without a recorded crash (bottom). 0.5 indicates no difference.

(Holm-adjusted 0.08). Our declared dependence-aware test weights each operator site equally (declared `e45b583`): mean rank 0.61, $p = 0.076$ (Holm 0.45). By its declared rule the result is *suggestive*: the effect is there on average, but it leans on a few sites with several crashed drones. A bootstrap that weights sites by their number of sets gives $p = 0.006$ (Supplement S5). The difference was clearest for aircraft-caused crashes ($\bar{r} = 0.67$, 46 sets) and similar in size but not significant for pilot-caused ones ($\bar{r} = 0.64$, 14 sets).

†*Screening value.* To test whether vibration could tell an operator which drones to inspect, we rebuilt the reference from the archive (declared `4961234`): 412 drones with no crash-rated log in the 31 strata of the 60 sets, with thresholds fixed on one random half and flag rates measured on the other. A threshold meant to flag 10% of drones flagged 13.7% of unseen ones and caught 24% of the crashed drones, a lift of 1.77× [1.00, 2.60] over the base rate (about 76 inspections per detected crash). By the declared rule, vibration ranking is *not shown to beat random inspection*. The 13.7% is the mean of the two split directions (10.7% and 16.8%); the bootstrap interval sits above it because resampling duplicates values in small strata and lowers their thresholds, and with leave-one-out thresholds, which overfit less, the flag rate was 12.9% and the lift 1.68× [0.94, 3.31] (declared check `e45b583`). Details and the operating points are in Supplement S5.

Flights flown after the crash†*.* Earlier flights were selected by calendar date. By GPS time, 85 of the 210 dated crash-day "earlier" flights were in fact flown after the crash (44–55% of those that could be ordered, depending on how flights without a usable GPS time are counted). Removing them left the declared conclusions unchanged and removed an apparent within-drone model effect (declared audit `0d38f5d`; Supplement S4).

5.7. External checks: a published checker, injected faults and a published trained detector (exploratory)

†Three checks outside our own pipeline test whether the patterns above are artefacts of our features, models or labels (Table 11).

A published log checker. †FlightMD [41] is an open-source, rule-based PX4 log checker whose author validated it on 50 Flight Review logs; it is not peer-reviewed. It turns findings from seven modules (oscillation, vibration, estimator, battery, GPS, parameters, motors) into a 0–100 health score. We ran its scoring code unchanged, without the parts that send the log's GPS position to external services (declared `fa5fb0b`; details in Supplement S8). Its score separated crashes from normal flights at AUC 0.79 on the whole log and 0.72 [0.58, 0.81] with the last 10 s removed, mostly through the estimator module; the crash window added less than the declared 0.05. Against each drone's own earlier flights it gave no warning (rank 0.48, $p = 0.66$), and as a pilot-or-aircraft verdict it reached a balanced accuracy of 0.52 [0.44, 0.61]. An independent tool therefore shows the same pattern as our pipeline: separation between aircraft, no warning within an aircraft, and no attribution.

A known-cause control: injected faults on real flights. †The archive cannot separate a tool that fails from labels that fail. RflyMAD [11] can: its real flights carry motor and sensor faults injected in the PX4 firmware on three multicopters, and each log records the injection, so cause and onset are known. We analysed 240 motor-fault, 197 sensor-fault and 51 fault-free real flights (declared `a9877c1`; Supplement S9). Two results bound the archive findings. First, the declared rule, built for crashes and here applied out of domain to faults the controller absorbed (mostly a 15–20% loss of motor efficiency), called only 3% of motor-fault flights aircraft-caused (balanced accuracy 0.50 [0.47, 0.52]); its archive failure therefore partly reflects signs that miss absorbed faults. Second, the study's flight-style features separated faulty from fault-free flights using only data from *before* the injection: AUC

Table 11: †External checks at a glance (AUC). Stricter splits hold out whole flights, drones or days; "before the fault" uses only data recorded before a fault was injected, where no fault exists.

Check	Task	Usual split → stricter split	Before the fault
FlightMD (published, untrained)	Crash vs normal, archive	0.79 (whole log) → 0.72 (last 10 s removed); no warning against the drone's own flights (rank 0.48)	0.47 (injected faults)
Flight-style features (logistic)	Faulty vs fault-free, RflyMAD	0.85 → 0.71 (days held out)	0.86 → 0.75 (days held out); same with length-matched windows
Published LeNet-5 detector (re-implemented)	Fault vs normal windows, RflyMAD	0.94 → 0.92 (flights) → 0.63 (days)	0.85 (flights) → 0.57 (days); length-matched 0.85 → 0.53

0.86 with flight-stratified folds and, in a declared check (`e45b583`), 0.75 with folds grouped by flight day and 0.79 grouped by flight pattern (permutation $p \leq 0.005$ for each, the smallest value 200 permutations can give); including 10 s of the fault added nothing (-0.04 [-0.08, 0.01] by day). FlightMD's health score showed no such separation (0.47 before the fault), so the separation comes from how the fault flights were flown (more tilt and tracking error before injection), not from the dataset as such. The fault-free flights were flown on only 4 of the 18 days, so a leave-day-out AUC pools folds that hold one class only; within those four days the pre-fault AUC was 0.96, 0.85, 0.32 and 0.68 (size-weighted mean 0.68; declared check `1e0bf0f`), so the separation is real on average but not consistent from day to day. The fault flights' pre-fault windows were also longer (median 36 s against 20 s; window length alone gives AUC 0.67), but with every flight cut to the same span after take-off the features still separated the classes (0.86 stratified, 0.75 with days held out, $p \leq 0.005$; declared `031fdd6`), so length does not explain the separation, while session differences may still contribute (Supplement S9).

A published trained detector. †FlightMD fits nothing, so it cannot show how training interacts with evaluation. We therefore re-evaluated a published *trained* detector on its own data: the RflyMAD authors' fault-diagnosis net-

work, an improved LeNet-5 convolutional network on 12 flight signals (velocity, angular velocity, acceleration and Euler angles), for which they report 98.2% accuracy on real single-motor faults in hover when trained on simulation data plus 10% of the real flights with domain adaptation [11]. No code or weights were released, so we re-implemented it from the published description, with the choices it leaves open fixed before any training (declared `1e0bf0f`; Supplement S11), and trained it on 3.2 s windows from the 240 motor-fault and 51 fault-free real flights. With windows split at random, as such results are usually reported, it reached 87.8% accuracy (AUC 0.94); with whole flights held out, 85.8% (AUC 0.92). Trained on real flights only, it did not reproduce the published accuracy even on a random split. With whole *days* held out it fell to 62.1% (AUC 0.626 against 0.943 for the random split, averaged over seeds, a drop of 0.32; on seed-averaged scores over all windows the drop is 0.33 [0.28, 0.38], 0.964 against 0.635). On windows recorded *before* the fault was injected, where no fault exists, it still separated fault flights from fault-free ones at AUC 0.85 with flights held out (permutation $p \leq 0.01$), and at 0.57 with days held out. Because only three of the four days that contain fault-free flights also contain motor-fault flights, we also scored the day-held-out detector within each of those days (declared `031fdd6`): fault versus normal AUC 0.83, 0.64 and 0.34 (mean 0.60), and before the fault 0.77, 0.48 and 0.38 (mean 0.51). With every flight's pre-fault windows limited to the same span after take-off, the detector still separated fault flights at 0.85 with flights held out ($p \approx 0.01$) but only 0.53 with days held out ($p = 0.12$). Within this re-implementation, then, the pre-fault separation is tied to the recording session: it survives holding out flights, even with matched windows, and disappears once days are held out. The flight-style features behave differently in degree, not in kind: their pooled day-held-out separation (0.75) survives length matching but varies from 0.32 to 0.96 between days. Neither shows a fault signal before the fault; both show that the result depends on how the data are split. Whether the authors' own model, trained with simulation data, behaves the same cannot be tested without its weights.

5.8. Expert diagnoses (Q4)

Of 148 forum threads describing a real crash, the expert attributed it to the aircraft in 94 (64%), to the pilot in 9 (6%), to both in 2, and was unclear or gave no diagnosis in 43 (29%). Tuning or configuration was the cause named most often (26 of 94; 14 of the 47 confirmed by the poster), followed

by GPS, compass or estimator problems (18) and motors, ESCs or propellers (16) (Fig. S2). Of 14 threads whose log carried an aircraft crash rating in the index, the expert agreed in 11 and was unclear in 3; none attributed the crash to the pilot. Download attempts across all 148 threads yielded 43 downloadable logs from 43 distinct threads (38 aircraft-fault and 5 pilot-fault; the remaining 105 had unhosted or expired external links). Of the 38 aircraft logs, exactly 35 were associated with checkable fault categories in Table S2 (12 tuning, 5 vibration, 5 GPS, 9 motor, 4 battery).

Against these 43 logs, the fixed rule called 14 of 32 decided aircraft-fault crashes pilot-caused, because the vehicle was in a pilot-flown mode at the crash and no failsafe fired, and 4 of 5 pilot-fault crashes aircraft-caused, because motors saturated or the accelerometer clipped at impact: a balanced accuracy of 0.38 (Bayesian 95% credible interval 0.25–0.61). Only half of the expert diagnoses were confirmed by the original poster, so this measures agreement with informed community opinion, not ground truth. With the last 10 s removed, none of five fault checks met its declared criterion (at least 50% of diagnosed logs, at most 10% of normal flights); the tuning check fell from 7 of 9 to 1 of 9 ($p = 0.031$, not significant after correction), showing that it depended on the crash window (Supplement S7, Table S2).

5.9. Language-model review panels (exploratory)

[†]Because language-model assistants are proposed for this task, we tested four small local models (3–4B parameters) as single reviewers and as a five-reviewer panel (Supplement S10). On 60 cases generated by the authors, the fixed rule was right in 45 and never wrong, and no panel matched it. On 97 real crash logs whose labels came from people who never saw the rule (35 forum-expert diagnoses and 62 from the blind relabelling, mapped to the panel's cause list before any run), no panel beat the rule either, while the rule named a cause different from the label in 62 of the 97. Because those labels are themselves contested (Section 5.2), this shows that the generated cases flattered the rule, not that the rule is wrong in 64% of real crashes. One compromised reviewer moved most panel verdicts, and the panel's verdict differed between two computers in 17 of 33 cases. We do not recommend such panels.

6. Design implications for fleet assurance

We built the findings into AeroFleet 2.0 (`https://github.com/Aditya Pathare46/aerofleet`), an open-source, ground-side assurance tool for PX4 and ArduPilot fleets. It reads the flight logs and configuration files an operator already has, never contacts the aircraft and makes no automatic decisions. Table 12 links each finding to a design choice.

Because logs did not separate pilot from aircraft causes (Q1–Q2), the tool lays out the events of a flight in time order, with the quality of each piece of evidence (GPS precision, sensor health, mode changes), and gives no pilot-or-aircraft verdict. Responding to the vibration finding (Q3), the tool shows each airframe's vibration over time and against comparable peers in the same fleet as context for an inspector, without a flag threshold presented as validated (measured against 412 non-crash drones, a top-decile threshold gave a lift of only 1.77× [1.00, 2.60]), and it issues no per-flight crash alarms. Every review, approval and configuration check is added to a tamper-evident record, signed with Ed25519 and tied to SHA-256 digests of the original log files, so the history of each airframe can be checked later.

6.1. Recommended reporting checklist for flight-log diagnostics

Table 13 turns the findings into a ten-item checklist for anyone who builds or evaluates log-based diagnostics, including regulators and accident investigators. Each item gives what to do and what went wrong in this study without it.

Table 12: From findings to design choices.

Finding	Result (section)	Design choice
Attribution failure	Across the telemetry features, rules, and models evaluated here, logs do not separate pilot from aircraft causes above chance (balanced accuracy 0.47 on held-out test, 0.381 on expert set, 95% CrI 0.25–0.61; §5.2, §5.8)	Present time-ordered event chronology and data quality metrics; abstain from automated fault verdicts.
Diagnostic leakage	Crash-window leakage was small for model AUC overall (0.014) but material among tilt-defined multicopter crashes (0.089 [0.019, 0.168]; §5.4), and present in one rule-based check (tuning sensitivity 7/9 $\rightarrow$ 1/9, exploratory; §5.3, §5.8)	Remove crash impact window ($m \geq 10\,\mathrm{s}$) and compare against aircraft operating baseline before diagnosing faults.
Between-drone vibration	Retrospective between-airframe association on older estimator metrics (mean rank 0.67; Holm $p = 0.016$ as declared, 0.08 clustered by airframe, 0.45 with operator sites weighted equally; measured screening lift 1.77$\times$ [1.00, 2.60]; §5.6)	Show each drone's vibration against fleet peers as context for inspection, without a flag threshold presented as validated.
Comparison choice	Grouping folds by vehicle was the only material factor in the factorial audit; comparing a drone with its own earlier flights lowered AUC by a further +0.113 [0.029, 0.212] on the same 67 crashes (§5.5)	Evaluate airframe health against its own flight history, not cross-sectional fleet averages.
Audit integrity	Language-model review panels added errors on generated and real cases, and one compromised reviewer moved most verdicts (§5.9)	Extract numbers from logs with fixed code, keep language models out of decisions, and record reviews in a signed log.

Table 13: Checklist for evaluating and reporting log-based diagnostics.

Item	What to do	Why: what happened here without it
1. Event-window removal	Remove ≥ 10 s prior to physical impact or ground tumble ($m \geq 10$ s)	Small for model AUC overall (Shapley share 0.014 [0.007, 0.023]) but material among tilt-defined multicopter crashes (0.089 [0.019, 0.168]; §5.4); present in one rule-based check (tuning 7/9 $\rightarrow$ 1/9; normal firing fell 4 of 11; Fisher $p = 0.057$). Confirm the crash moment before cutting; apply tilt rules to multicopters only.
2. Airframe- and session-grouped folds	Partition cross-validation folds by vehicle, and by day or session when vehicles are few	Stops a model from recognising individual vehicles: grouping lowered AUC from 0.882 to 0.826 on the 287-flight sample and was the only material factor in the factorial audit (0.062 [0.034, 0.087]; §5.5).
3. Longitudinal baseline	Evaluate cases against the airframe's own pre-incident flights	Separates the flight from the aircraft. Order flights by GPS time, not date: 85 of 210 crash-day "earlier" flights in this archive were flown after the crash; without them the within-drone model gave at most a weak signal (0.582 [0.525, 0.660]; §5.6).
4. Duration reporting	Report window durations of cases and comparisons; use duration matching only as a sensitivity analysis	Aborted or short crash flights create duration differences. A caliper lift of +0.110 [+0.028, +0.205] vanished once post-crash flights were removed (+0.086 [−0.064, +0.184]; §5.6).
5. Two-class expert ground truth	Validate against multi-coder independent diagnoses with reported agreement ($\kappa \geq 0.70$)	Uploader labels are contestable; report how well independent raters agree, for both pilot-fault and aircraft-fault cases (§5.8).
6. Operational base-rate calibration	Measure flag rates on held-out non-case units, then report exact Bayes PPV, lift and inspections per event at fleet base rates	At base rates of 1% or less a high AUC still means many false alarms; assumed flag rates overstated lift here (3.3× assumed, 1.77× measured; §5.6).
7. Prospective protocol registration	Pre-specify hypotheses, sample funnels, features, models, seeds, and pass marks in version control	Guards against choosing analyses after seeing results, and makes exploratory additions visible (§4.1).
8. Persistent open artifact deposit	Deposit raw logs, extraction code, and derived feature tables in public archives	Lets others check and repeat the analysis.
9. Label audit†	Check that each label class actually contains the event it names (e.g. share of logs with a crash-like signature, per class)	61% of crash-rated multicopter logs show no crash-like tilt, and the share differs by responsibility label (23% vs 45%), so a classifier could separate labels without learning cause (§5.4).
10. Pre-onset baseline†	Where onset is known, report the same classifier on data before the onset	In one injected-fault dataset, flight-style features reached AUC 0.75 before the fault with folds grouped by day, and the fault added nothing (§5.7); how large this effect is elsewhere is unknown, which is why it should be reported.

7. Discussion

Attribution. A declared rule, its held-out replication, cross-validated models, a published log checker and expert-diagnosed logs all failed to separate pilot-caused from aircraft-caused crashes with these features and labels, including on the 108 multicopters alone. With 36 pilot-labelled crashes, only an AUC of about 0.72 could be detected with 80% power, and an operational verdict would need about 0.8; the result therefore bounds the evidence in this archive rather than showing attribution to be impossible. †The independent relabelling does not settle whether the labels are to blame. The reviewer agreed with the uploaders no better than chance, and the tools also failed against the reviewer's labels, but those labels come from one reviewer working from plots, so the two label sets are both contestable and label noise cannot be excluded. What the data do show is that neither label set is recovered from the logs, and that the uploader's rating partly tracks whether the log shows a tilt-type crash. Accidents are multi-causal [7]; a log records the combined outcome of pilot inputs, disturbances and vehicle response, and tools that issue automatic responsibility verdicts should be evaluated against independent investigations before they are used.

Leakage and confounding. Most of the apparent separation between crash flights and normal flights came from differences between aircraft and from how the evaluation was built, not from the crash. Grouping folds by vehicle was the only isolable source of optimism in the factorial audit (0.062 of a 0.212 drop); separation against other aircraft persisted (0.67), and only comparison with the same drone's own earlier flights brought it to chance. Crash-window leakage was small overall (0.035) but material among multicopter crashes with a crash-like tilt (0.089 [0.019, 0.168], post hoc), and it grew when tilt and stick features were removed, so it is not just the defining tilt being cut. Two external checks point the same way. FlightMD, which fits nothing and so cannot benefit from fold leakage, still separated crash flights from other drones' flights (0.72) through estimator health and lost the signal against each drone's own flights. A re-implemented published detector fell from AUC 0.94 to 0.63 when whole days were held out, and its separation of fault flights before the fault (0.85 with flights held out) fell to 0.57 with days held out. On the same flights, flight-style features (tilt and tracking error) separated faulty from fault-free flights before the fault (AUC 0.75 even with folds grouped by flight day), while FlightMD's score did not; such separation is therefore

specific to the features, not inevitable, but a high fault-versus-normal AUC is weak evidence of a fault signal unless a pre-onset or same-aircraft baseline is reported.

Archive records[†]*.* The tilt rule that usually fixes the crash moment finds no crash-like tilt in 61% of crash-rated multicopter logs. Three raters agreed that the rule's crash moment is right when tilt-defined and mostly wrong otherwise, but their estimates of how many crash-rated multicopter logs contain a crash range from 50% to 82% (majority of three 67%). The rule therefore misses roughly 7–50% of crashes depending on the rater, which is a different statement from the logs lacking them. The share of logs with a tilt event also differs by responsibility label (23% of pilot-rated against 45% of aircraft-rated, adjusted odds ratio 2.1), so a classifier could separate these labels by detecting tilt-type crashes, whether because recordings of pilot-caused crashes stop earlier or because those crashes involve less tilt. A tilt rule is a crash criterion only for multicopters, and vehicle type must come from the archive index, because older logs lack the flag that would exclude fixed-wing and VTOL aircraft. Dates cannot order flights within a day. These are properties of the records, not of a particular model, and they apply to any study that uses the archive's ratings as labels [12].

Drone-level risk. The one positive finding concerns airframes, not flights. Drones that later crashed vibrated more than comparable drones on ordinary flights, but the effect is suggestive rather than established: $p = 0.012$ when airframes are resampled and $p = 0.076$ when each of the 23 operator sites counts once. Two alternative explanations remain open: experimental or poorly tuned airframes may vibrate throughout their lives and crash more often during testing, and the effect was seen on the older estimator metric only, with an inconclusive and underpowered replication on the newer one. As a screening tool the signal is weak: with flag rates measured on held-out non-crash drones, a top-decile vibration threshold gave a lift of 1.77× [1.00, 2.60], rather than the 3.3× a flag rate assumed in advance would suggest. Vibration is therefore a candidate for prospective testing, not a validated inspection rule or a crash warning.

Implications. Section 6 turns these findings into a design that reviews evidence instead of issuing verdicts, and Table 13 into a reporting checklist. For operators and investigators: treat automatic pilot-or-aircraft verdicts from logs with caution; use logs to build a time-ordered record of events for

a human investigator; keep records per aircraft over time, so that a drone can be compared with its own history and with comparable airframes [1, 2]; and, when evaluating any log-based diagnostic, remove the crash window, group folds by vehicle and compare against the same aircraft's history. Small language-model review panels added errors and could be steered by one compromised reviewer, both on generated and on real cases (Section 5.9).

Future work. Two studies were declared in the repository before any data were collected: a six-month prospective test in partner fleets, in which the monthly vibration ranking is written to the signed record before outcomes occur, and an evaluation with human operators. A set of real crashes with causes verified by independent investigators is the main missing resource; sequence models or frequency-domain features may find transient precursors that whole-window summaries miss.

8. Threats to validity

Archive self-selection. The public archive reflects voluntary uploads from hobbyists, researchers and commercial operators, which favours unusual flights, tuning sessions and unexplained incidents. Crash and normal flights differ in duration, firmware era and flight envelope, which motivated the within-drone and matched designs. Results may not transfer to ArduPilot or proprietary autopilots.

Labels. Uploader ratings are the uploaders' own judgements, and normal flights are self-rated; controls without a crash rating may have crashed unnoticed, which makes the between-drone test conservative. †The independent relabelling was done by one engineering student who is not an author, working only from the blind plot pack (no log identifiers, dates, notes, ratings or Flight Review links) and asked not to open the repository, which cannot be verified; the same person was the third crash-moment rater and supplied 62 of the 97 language-model labels. These three checks therefore rest on one non-author whose reliability could not be measured; a second independent rater would remove that concentration. Their near-zero agreement with the uploaders therefore shows that the labels are contestable, not which side is right. The 43 expert-diagnosed logs reflect informed community opinion, confirmed by the original poster in only about half of the threads.

Confounding and dependence. Matching on hardware, PX4 version and vehicle type leaves propellers, payload and flight style uncontrolled. Operators are not recorded; the location proxy used to cluster by site (first GPS fix, 2 km) is not validated against known operators, and 20 airframes had no GPS fix.

Measurement. The crash moment is estimated, and the tilt rule misses roughly 7–50% of crashes depending on the rater, an estimate that rests on 15 reviewed tilt-defined logs; the within-drone vibration result did not depend on the tilt threshold (45°: $\bar{r} = 0.46$; 60°: 0.48; 75°: 0.50). Whole-window summary features may miss short precursors. Crash windows end at the tilt crossing while normal windows end at disarm, after the landing; ending normal flights at touchdown instead, as a pseudo crash moment, left the leak drop unchanged (0.037 against 0.035 overall, 0.113 against 0.114 for tilt-defined multicopters; declared check `e45b583`).

Protocols and registration. Protocols were declared and committed to version control in advance of analysis (confirmatory tests committed 20–24 October 2026: `63bc72e`, `af6fab2`, `4475c8c`, `0c8ee47`, `f911bcf` and `d5d56c7`, `8b417d2`, `d7a6574` on 20 and 23 October; `2148644` and `21af373` on 24 October; times in Table 4), and archived snapshots were deposited in Software Heritage (`swh:1:rev:bad06ff1256865c7e63d44389aac174e0281291d`, `swh:1:rev:bbdd0420de4547ac7d67b84165292a46237b604d`, `swh:1:rev:624dd694e0d207dd9c44b1c9550faaf04453bc56`, and `swh:1:rev:3a9aa89417fa9ea2a2334e3a4794e2358fb713ee`); however, because the earliest Software Heritage capture dates from 25 October 2026, commit timestamps prior to 25 October in self-hosted git remain author-controlled. The audits added after 25 October (from `3c34530` onwards, 26 October to 3 November 2026) have no independent archive yet and will be archived with the Zenodo release; distinguishing declared protocols from formal third-party timestamped registries (such as the Open Science Framework). One amendment (lifting the within-drone firmware filter) was data-dependent after observing sample starvation ($n = 12$ under the initial ≥ 1.10 filter).

Power. Several comparisons are small: 36 pilot-labelled crashes (27 among multicopters), 14 pilot-caused matched sets, 19 sets on the newer vibration metric and 2–9 logs per expert fault check, so null results there are not evidence of no effect. Detecting the between-drone effect on the newer metric

with 80% power would need 55 sets if the discovery estimate is unbiased and about 247 if it is inflated, more than the archive holds.

External checks[†]. The injected-fault control uses three airframes flown by one team, with mild faults that rarely end in a crash; folds grouped by flight day reduced the pre-fault AUC from 0.86 to 0.75 but did not remove it, and the fault-free flights were flown on only 4 of the 18 days, so session differences may contribute. The declared rule was built for crashes, so its result there is an out-of-domain check of faults the controller absorbed, not evidence that it would miss a fault that ends in a crash. FlightMD is not peer-reviewed. The language-model test used small local models and labels that are themselves contested.

The tool. AeroFleet 2.0 is an open-source evidence-review tool whose operational safety benefit has not been shown in the field; prospective evaluation in fleets and with human operators remains future work.

9. Conclusion

Across 590 crash-rated logs from a 479,547-flight PX4 archive, three findings stand.

First, evaluation design creates much of the apparent skill of log-based crash diagnostics. Across 32 settings the apparent AUC fell by 0.21, of which 0.06 came from grouping folds by vehicle, and comparing each drone with its own earlier flights removed the rest. The pattern held for an untrained published checker, for injected faults of known onset and for a re-implemented published detector, which fell from AUC 0.94 to 0.63 once whole days were held out.

Second, the largest public archive of real crash logs needs auditing before its records are used as labels: the usual tilt rule misses roughly 7–50% of crashes depending on the rater, the uploader's rating partly tracks whether the log shows a tilt-type crash, an independent relabelling disagreed with the uploaders, and dates cannot order flights within a day.

Third, these logs support evidence review, not verdicts. No rule, model, published checker or expert comparison separated pilot-caused from aircraft-caused crashes, crash flights showed no warning against the same drone's history, and the one positive signal, higher vibration in drones that later crashed, is suggestive ($p = 0.076$ with operator sites weighted equally) and was not shown to beat random inspection.

Log-based diagnostics should therefore be evaluated with vehicle- and session-grouped folds, against each aircraft's own history and with flights ordered by GPS time, and fleet tools should present evidence rather than issue verdicts. The checklist in Table 13 sets out these controls, and AeroFleet 2.0 implements the evidence-first design as an open-source tool.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. The authors develop the open-source AeroFleet 2.0 tool described in Section 6; it is not commercial.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used an AI assistant (Claude, Anthropic) to write analysis code, draft and revise text, and assist the recording of forum diagnoses. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Data availability

Selection scripts, analysis code, the declared hypotheses with their commit history, derived features, expert labels (without forum text), and the source code of the AeroFleet 2.0 tool are available at `https://github.com/AdityaPathare46/aerofleet`; a DOI-archived release will be provided on acceptance. Raw logs are public on PX4 Flight Review and are referenced by identifier. The RflyMAD real flights were taken from the dataset's Kaggle copy (`xianglile/rflymad`) under its non-commercial terms and are not redistributed. Take-off locations used as an operator proxy were computed in memory and are neither stored nor released; only cluster sizes are reported. Returned review workbooks are not released; derived labels are.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

[1] European Commission, Commission Implementing Regulation (EU) 2019/947 of 24 May 2019 on the rules and procedures for the operation of unmanned aircraft, Official Journal of the European Union, L 152, 11 June 2019 (2019).

[2] European Union Aviation Safety Agency, Easy Access Rules for Unmanned Aircraft Systems (Regulations (EU) 2019/947 and 2019/945), EASA, Cologne, revision of June 2026 (2026).

[3] Ministry of Civil Aviation, Government of India, The Drone Rules, 2021, G.S.R. 589(E), Gazette of India, 25 August 2021 (2021).

[4] Federal Aviation Administration, 14 CFR Part 107: Small Unmanned Aircraft Systems, Electronic Code of Federal Regulations, as of 21 October 2026 (2026).

[5] V. S. A. Duvvuru, B. Zhang, M. Vierhauser, A. Agrawal, LLM-agents driven automated simulation testing and analysis of small uncrewed aerial systems, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2025, pp. 385–397. doi:10.1109/ICSE55347.2025.00223.

[6] S. Silalahi, T. Ahmad, H. Studiawan, F. Breitinger, DroPTC: Sentence-level drone flight log forensics using contrastive learning and explainable AI, Forensic Science International: Digital Investigation 56 (2026) 302051. doi:10.1016/j.fsidi.2026.302051.

[7] D. A. Wiegmann, S. A. Shappell, A Human Error Approach to Aviation Accident Analysis: The Human Factors Analysis and Classification System, Ashgate, Aldershot, 2003.

[8] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman, Leakage in data mining: Formulation, detection, and avoidance, ACM Transactions on Knowledge Discovery from Data 6 (4) (2012) 15:1–15:21. doi:10.1145/2382577.2382579.

[9] S. Kapoor, A. Narayanan, Leakage and the reproducibility crisis in machine-learning-based science, Patterns 4 (9) (2023) 100804. doi:10.1016/j.patter.2023.100804.

[10] A. Keipour, M. Mousaei, S. Scherer, ALFA: A dataset for UAV fault and anomaly detection, The International Journal of Robotics Research 40 (2–3) (2021) 515–520. doi:10.1177/0278364920966642.

[11] X. Le, B. Jin, G. Cui, X. Dai, Q. Quan, RflyMAD: A dataset for multicopter fault detection and health assessment, The International Journal of Robotics Research 44 (7) (2025) 1081–1092. doi:10.1177/02783649241305153.

[12] A. Kabaoglu, S. Sariel, UAV-SEAD: State estimation anomaly dataset for UAVs, arXiv preprint https://arxiv.org/abs/2602.13900 (2026).

[13] G. Wild, J. Murray, G. Baxter, Exploring civil drone accidents and incidents to help prevent potential air disasters, Aerospace 3 (3) (2016) 22. doi:10.3390/aerospace3030022.

[14] Air Accidents Investigation Branch, Annual Safety Review 2025, Tech. rep., AAIB, Department for Transport, Farnborough, UK, published 4 June 2026 (2026).

[15] L. Meier, D. Honegger, M. Pollefeys, PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms, in: 2015 IEEE International Conference on Robotics and Automation (ICRA), 2015, pp. 6235–6240. doi:10.1109/ICRA.2015.7140074.

[16] PX4 Development Team, Flight Review log archive and pyulog ULog parser, https://review.px4.io; pyulog v1.2.4, https://github.com/PX4/pyulog; BSD-3-Clause (2026).

[17] J. Wei, H. Li, Y. Li, Y. Zhao, D. Yao, B. Lu, X. Ye, S.-K. Im, Y. Wang, X. Yang, AeroTSBoost: Temporal-statistical boosting for real-world UAV telemetry anomaly mining, arXiv preprint https://arxiv.org/abs/2605.25639 (2026).

[18] K. Kose, J. Wing, N. A. Kose, C. Guadarrama-Trejo, A. Sowers, A. Rasheed, A UAV testbed for diagnosing hardware vulnerabilities: Quantifying sim-to-real discrepancies in PX4 flight logs, Sensors 26 (10) (2026) 3188. doi:10.3390/s26103188.
URL https://www.mdpi.com/1424-8220/26/10/3188

[19] D. Wang, S. Li, G. Xiao, Y. Liu, Y. Sui, P. He, M. R. Lyu, An exploratory investigation of log anomalies in unmanned aerial vehicles, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), 2024, pp. 2593–2605. doi:10.1145/3597503.3639186.

[20] R. Puchalski, W. Giernacki, UAV fault detection methods, state-of-the-art, Drones 6 (11) (2022) 330. doi:10.3390/drones6110330.

[21] E. Khalastchi, M. Kalech, On fault detection and diagnosis in robotic systems, ACM Computing Surveys 51 (1) (2018) 9:1–9:24. doi:10.1145/3146389.

[22] E. Zibaei, R. Borth, Building causal models for finding actual causes of unmanned aerial vehicle failures, Frontiers in Robotics and AI 11 (2024) 1123762. doi:10.3389/frobt.2024.1123762.

[23] D. Wang, S. Li, G. Xiao, Y. Liu, Y. Sui, An exploratory study of autopilot software bugs in unmanned aerial vehicles, in: Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2021, pp. 20–31. doi:10.1145/3468264.3468559.

[24] T. Kim, C. H. Kim, A. Ozen, F. Fei, Z. Tu, X. Zhang, X. Deng, D. J. Tian, D. Xu, From control model to program: Investigating robotic aerial vehicle accidents with MAYDAY, in: Proceedings of the 29th USENIX Security Symposium, USENIX Association, 2020.
URL https://www.usenix.org/conference/usenixsecurity20/presentation/kim

[25] M. Seong, W. Yun, C. Lee, S. Lee, J. Park, Flight log analysis and recovery for open-source drones: Focusing on ArduPilot, PX4, and Betaflight, Computers and Electrical Engineering 135 (2026) 111144. doi:10.1016/j.compeleceng.2026.111144.
URL https://linkinghub.elsevier.com/retrieve/pii/S0045790626002168

[26] R. Hakani, A. Rawat, A. Kumar, Enhancing UAV safety through battery health monitoring and flight log-based crash diagnostics, IEEE Sensors Reviews (2025). doi:10.1109/SR.2025.3597582.

[27] E. Mantas, C. Patsakis, GRYPHON: Drone forensics in dataflash and telemetry logs, in: Advances in Information and Computer Security, Lecture Notes in Computer Science, Springer, 2019, pp. 377–390. doi:10.1007/978-3-030-26834-3_22.

[28] E. Petritoli, F. Leccese, L. Ciani, Reliability and maintenance analysis of unmanned aerial vehicles, Sensors 18 (9) (2018) 3171. doi:10.3390/s18093171.

[29] A. P. Tvaryanas, W. T. Thompson, S. H. Constable, Human factors in remotely piloted aircraft operations: HFACS analysis of 221 mishaps over 10 years, Aviation, Space, and Environmental Medicine 77 (7) (2006) 724–732.

[30] H. Li, J. Ma, Z. Wang, L. Chen, R. Xu, J. Ai, UFA: An LLM-driven UAV forensic intelligent agent for low-altitude public security, Expert Systems with Applications 327 (2026) 132859. doi:10.1016/j.eswa.2026.132859.
URL https://linkinghub.elsevier.com/retrieve/pii/S0957417426017720

[31] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, Improving factuality and reasoning in language models through multiagent debate, in: Proceedings of the 41st International Conference on Machine Learning (ICML), Vol. 235 of PMLR, 2024, pp. 11733–11763.
URL https://proceedings.mlr.press/v235/du24e.html

[32] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79–90. doi:10.1145/3605764.3623985.

[33] J. P. Vieira, V. A. Bauler, R. K. Rosa, D. Silva, Towards a more realistic evaluation of machine learning models for bearing fault diagnosis, arXiv:2509.22267 (2025). doi:10.48550/arXiv.2509.22267.

[34] B. Frénay, M. Verleysen, Classification in the presence of label noise: A survey, IEEE Transactions on Neural Networks and Learning Systems 25 (5) (2014) 845–869. doi:10.1109/TNNLS.2013.2292894.

[35] E. A. Stuart, Matching methods for causal inference: A review and a look forward, Statistical Science 25 (1) (2010) 1–21. doi:10.1214/09-STS313.

[36] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, D. T. Mellor, The preregistration revolution, Proceedings of the National Academy of Sciences 115 (11) (2018) 2600–2606. doi:10.1073/pnas.1708274114.

[37] J. A. Hanley, B. J. McNeil, The meaning and use of the area under a receiver operating characteristic (ROC) curve, Radiology 143 (1) (1982) 29–36. doi:10.1148/radiology.143.1.7063747.

[38] H. B. Mann, D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, The Annals of Mathematical Statistics 18 (1) (1947) 50–60. doi:10.1214/aoms/1177730491.

[39] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, New York, 1993.

[40] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.
URL https://www.jstor.org/stable/4615733

[41] P. Behl, FlightMD: open-source diagnostic tool for PX4, ArduPilot and MAVLink flight logs, GitHub repository, commit 306d0cb, MIT licence; accessed 29 October 2026 (2026).
URL https://github.com/Praddyx15/FlightMD